

13/10/2023

parallèle :

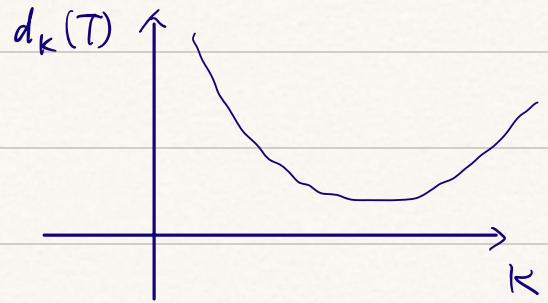
principe général d'organisation

→ tâche T $d_1(T)$ avec 1 personne
↓
réduire $d_1(T)$ K

idéal $\frac{d_1(T)}{K} = d_K(T)$

parallélisme :

- réduire durée
- augmenter débit



$S_1: C \Leftarrow A + B$ $S_2: D \Leftarrow E * F$ $S_1 \parallel S_2 ?$ non.

freins / obstacles au parallélisme :

- ressources
- dépendances

Fortran : identifiants $\neq \Rightarrow$ cases mémoires $\neq$

C : pas de garantie

statement : read, operation, write

S_1 puis S_2 même cases de mémoire

$S_2 \backslash S_1$	R	W
R	read after read RAR	read after write RAW ⚠
W	write after read WAR ⚠	write after write WAW ⚠

⚠ dépendance de données (Data)

$$D = E + F \quad G = E * H$$

$$D = E + F \quad G = D * H$$

$$D = E + F \quad E = G * H$$

$$D = E + F \quad D = G * H$$

artificiel

$$A = G * H$$

→ pas de problème de ressource, pas de dépendance

do $i = 1, 100$

$$C(i) = A(i) + B(i)$$

$$C(i) = C(i-3) + B(i) \quad \leftarrow \text{non parallélisable}$$

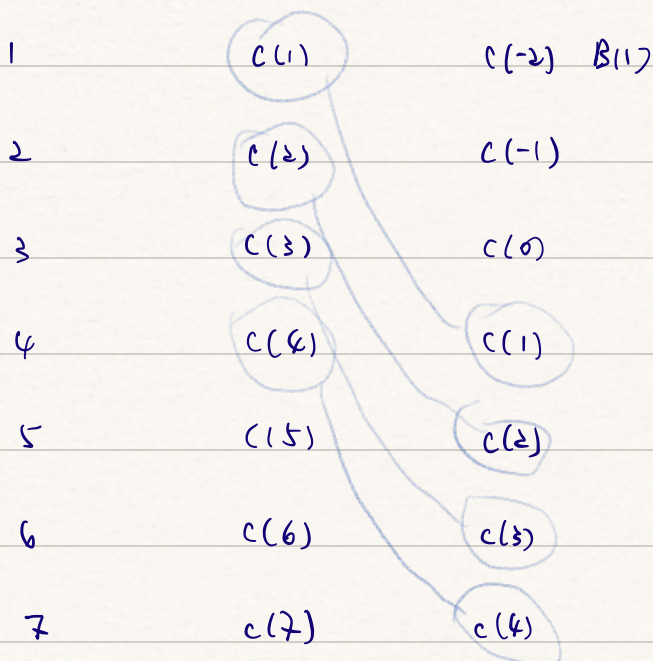
enddo

$$\begin{cases} A(1:100) \cap C(1:100) = \emptyset \\ B(1:100) \cap C(1:100) = \emptyset \end{cases}$$

condition suffisante

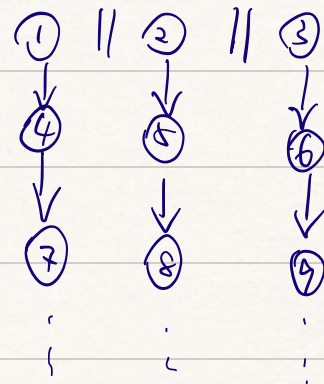
$$B(1:100) \cap C(1:100) = \emptyset$$

W R



dépendance entre itérations

chaînes en parallèle



ASM :

→ case mémoire $f(R_2) \leftarrow (R_2) \times k$

→ registers

$$R_7 = R_1 + R_3$$

$$R_9 = R_1 + R_{12}$$

$$R_7 = R_1 + R_3$$

$$R_1 = R_9 * R_{12}$$

physical register (100 → 200)

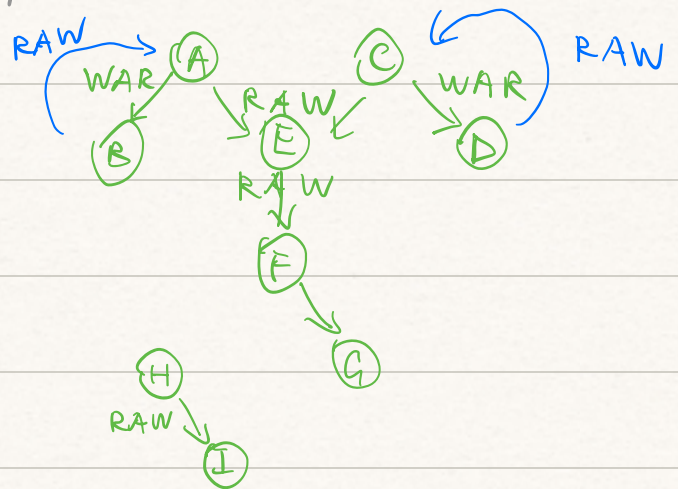
virtual register (16/32) Itanium 128

4KB virtual pages $\xrightarrow{\text{mémoire virtuelle}}$ physical 4 ~ 30 cycles
>>

硬件、材料限制

```
A loop: load A(i), fp1
B      inc @A
C      load B(i), fp2
D      inc @B
E      fadd(d) fp1, fp2, fp3
F      store fp3, C(i)
G      inc @C
H      dec lc
I      bgtz lc, loop
```

floating point register



dépendance de contrôle

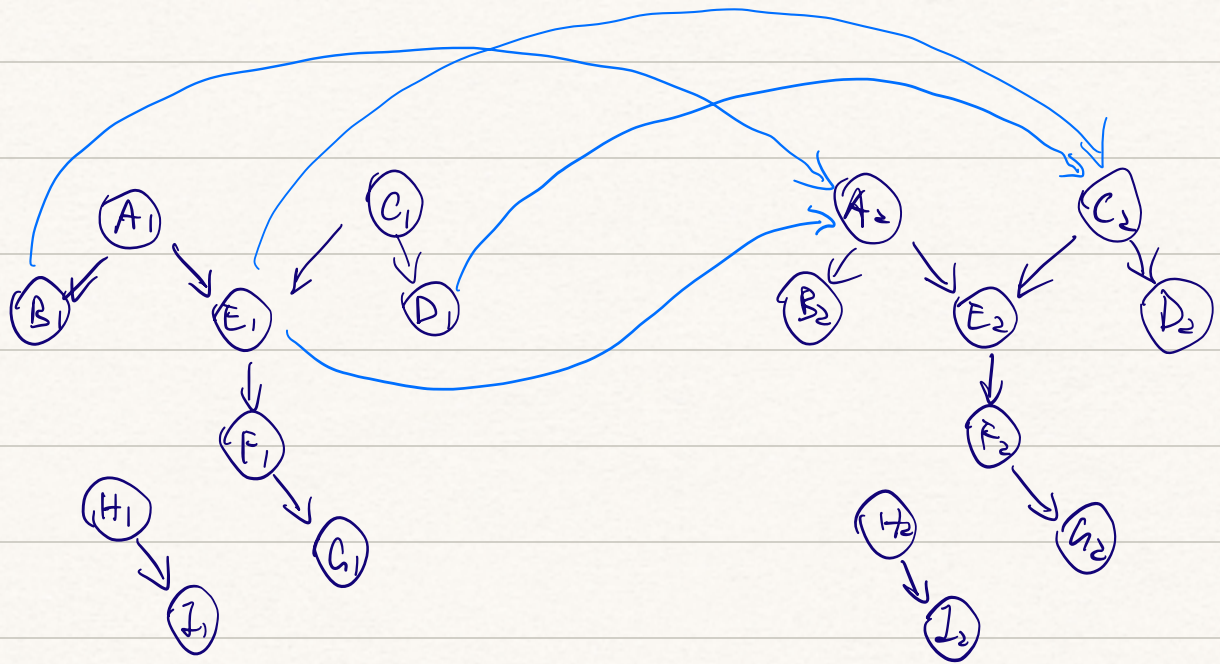
branch greater than zero

blo

branch lower than one

rip

iteration :



instruction level parallelism

27/10/2023

objets réguliers 1D :

ensemble des cases mémoires (pas d'ordre mais construit avec)

@DEP: adresse départ } @ DEP + (k-1) * S k ∈ [1, N]
S: stride }
#N: nb d'éléments }

@ DEP, @ DEP + S, @ DEP + 2S, ...

1, 5, 9, 13, 17 @ DEP = 1, S = 4, #N = 5

1, 8, 15, 22, 29, 36, 43 S = 7

structure 2D :

row	length = RL						
1	6	7	8	9	10	11	12
2	13	14	15	16	17	18	19
3	20	21	22	23	24	25	26
4	27	28	29	30	31	32	33
5	34	35	36	37	38	39	40
6	41	42	43	44	45	46	47

(14, 1, 4)

(19, 7, 5)

$A(i, j)$ $A[i][j]$. . .

→ rangement par ligne

(6, 8, 6)

1 2 3 4 5 6 7

$A(3, 3)$

$@ A(i, j) = @ A(1, 1) + (i-1) * 7 + (j-1)$

6 + 2 * 7 + 2

objet régulier 2D :

{ @ DEP
SL stride ligne + SC stride colonne
#NL + #NC

@ DEP + (i-1) * SL + (j-1) * SC j puis i
i ∈ [1, NL], j ∈ [1, NC]

rangement par colonne $\leftarrow$

10	15	20	25	30
11	16	21	26	31
12	17	22	27	32
13	18	23	28	33
14	19	24	29	34

accès régulier $\rightarrow$ l'ordre des objets réguliers

do $i = 1, 100$

$$A(3 \times i) = B(i + 12) + C(-5 \times i - 1)$$

enddo

A

B

C

1

A(3)

B(13)

C(-6)

2

A(6)

B(14)

C(-11)

3

A(9)

B(15)

C(-16)

4

A(12)

B(16)

C(-22)

5

A(15)

B(17)

C(-26)

do $i = 1, 100$

$$A(i \times i) = 2.0 \quad \text{float}$$

enddo

A(i x i)

1

A(1)

2 $A(4)$

3 $A(9)$

4 $A(16)$

5 $A(25)$

do $i = 1, 6$

$A(i) = B(C(i))$

enddo

$C()$

$C()$

$C()$

1 3

12

5

2 4

19

7

3 5

21

3

4 6

3

9

5 7

5

1

6 8

7

11

↑
objet régulier mais
pas d'accès régulier à $B(C(i))$

double position

$A(N \times N)$ DP 8 B / élément

$N = 10^6$

$|A(N \times N)| = 10^6 \times 10^6 \times 8B$

matrice creuse nb d'élément $\neq 0$ par ligne ≤ 20

$$y = Ax \quad x, y \in \mathbb{R}^n$$

do $i = 1, N$

do $j = 1, N$

$$y(i) = y(i) + A(i, j) * x(j)$$

enddo

enddo

info ntile $20 \times 10^6 \times 8B$ valeurs

$20 \times 10^6 \times (4B + 4B)$ indices

40 VS 10^6

A

	1.1	0.6		0.8		
	3.2					
4.8		4.9				
	8.2	7.4	7.5			
6.2	1.0					
2.0						
3.2	4.8		6.7			
1	2	3	4	5	6	7

VAL valeurs

1.1	0.6	0.8
3.2	0	0
4.8	4.9	0
8.2	7.4	3.5
6.2	1.0	0
2.0	0	0
3.2	4.8	6.7

IND indices

2	4	7
2	2	2
1	4	2
3	5	6
1	3	2
1	2	2
1	3	6

CSR
compress storage row

$$VAL(i, j) = A(i, IND(i, j))$$

do $i = 1, N$

do $j = 1, 3 \text{ ou } 20$ (max de nb val. $\neq 0$)

$$y(i) = y(i) + VAL(i, j) * x(IND(i, j))$$

accès régulier
non régulier

enddo

enddo

VAL : $20 \times 10^6 \times 8B$

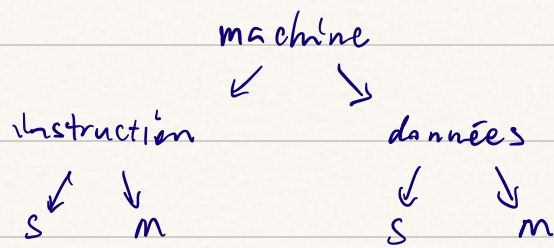
IND : $20 \times 10^6 \times 8B$

VAL 1,1 0,6 0,8 3,2 4,8 4,9 8,2 3,4 7,5 6,2 1,0 2,0 3,2 4,8 6,7

RIND 1 1 1 2 3 3 4 4 4 5 5 6 7 7 7

CIND 2 4 7 2 1 4 3 5 6 1 3 1 1 3 6

classification



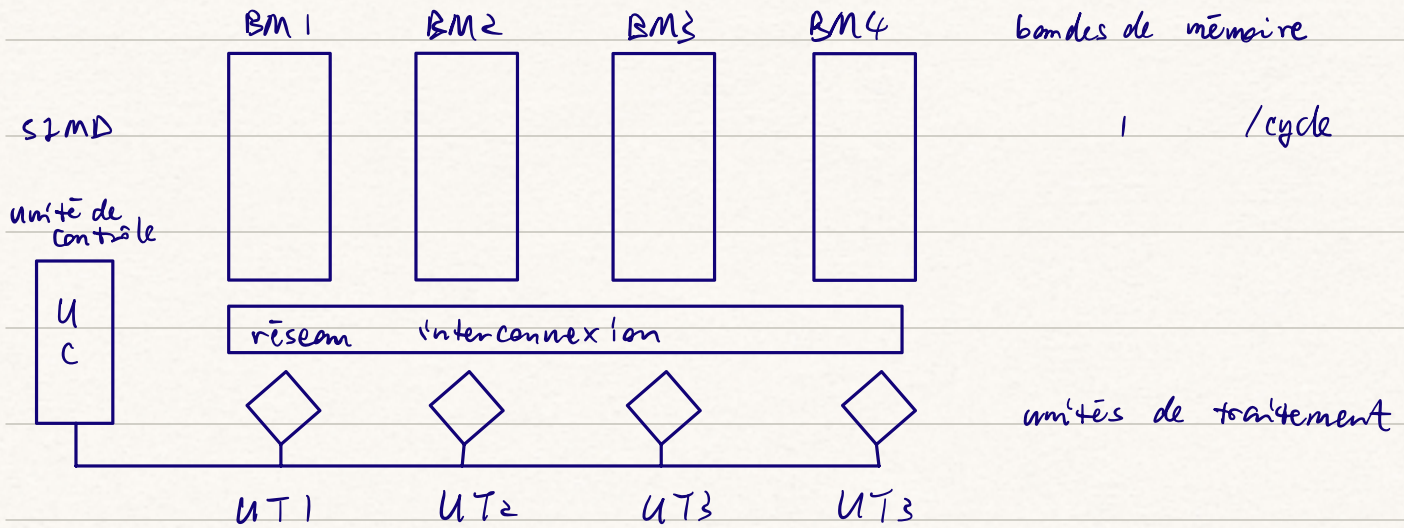
S: single M: multiple

I \ D	S	M
	classique	
S	SISD	MISD
M	SIMD	MIMD

do $i \geq 1, 100$

$$C(i) = A(i) + B(i)$$

enddo



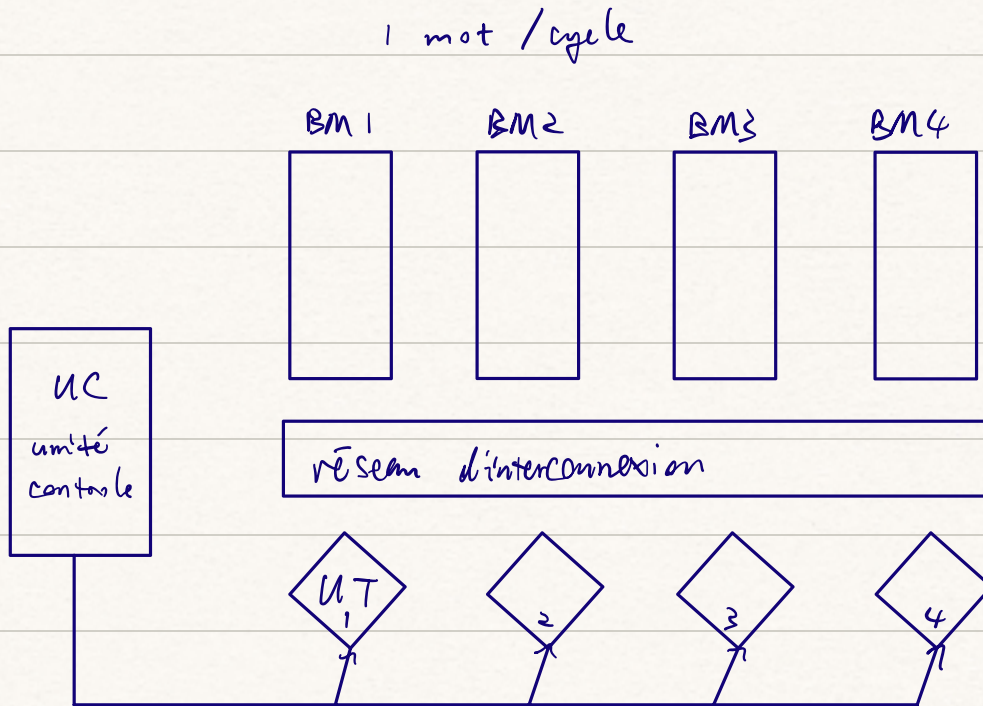
$100 = 25 \times 4$ itérations consécutives

UC	UT1	UT2	UT3	UT4	
load A	A(1)	A(2)	A(3)	A(4)	lock step mode
load B	B(1)	B(2)	B(3)	B(4)	
add	+	+	+	+	
store	C(1)	C(2)	C(3)	C(4)	

13/11/2023

unité traitement = proc - controle

exec = 12MSY



do $i = 1, 100$

$C(i) = A(i) + B(i)$

end do

$C(1:100) \cap A(1:100) = \emptyset$

$C(1:100) \cap B(1:100) = \emptyset$

loop (i): load $A(i), FP_1$

load $B(i), FP_2$

add FP_1, FP_2, FP_3

store $FP_3, C(i)$

incrémenter inc @ $A \ B \ C$

décrémenter dec loop C ^{loop compteur}

^{branch} greater than BGT loop 1

"packet = tranche"

25 packets de 4 iter consécutives

$p_1: 1 \ 2 \ 3 \ 4$

p₂ : 5 6 7 8

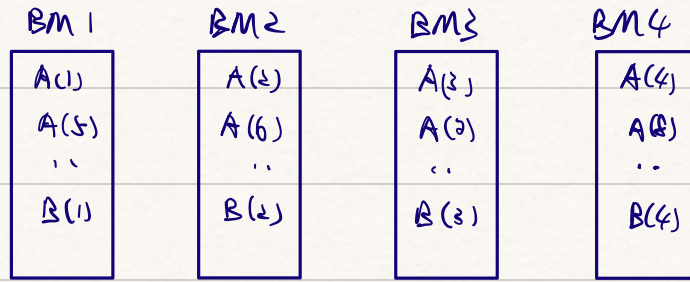
:

UC	BM UT ₁	BM UT ₂	BM UT ₃	BM UT ₄	
load	A(1)	A(2)	A(3)	A(4)	lock step mode
load	B(1)	B(2)	B(3)	B(4)	
add	+	+	+	+	
store	C(1)	C(2)	C(3)	C(4)	
INC@	INC +4	INC +4	INC +4	INC +4	frame 1
DEC					
BGT					
load	A(5)	A(6)	A(7)	A(8)	
load	B(5)	B(6)	B(7)	B(8)	
add	+	+	+	+	
store	C(5)	C(6)	C(7)	C(8)	
INC@					
DEC					
BGT					



1 inst / cycle

meanx :



$A(i) \rightarrow \text{BM } k \quad \text{t.q.} \quad k \equiv i[4]$

(pas de rangement entre A et B)

do $i = 1, 100, 2$

$$C(i) = A(i) + B(i)$$

end do

$$C(1:100) \cap A(1:100) = \emptyset$$

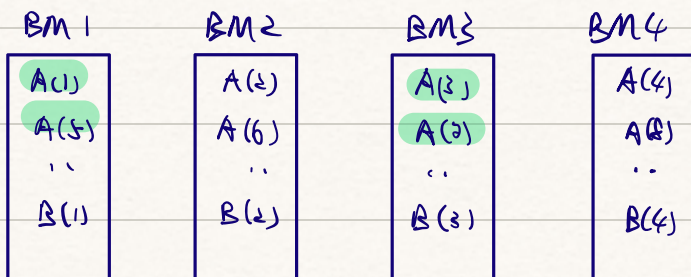
$$C(1:100) \cap B(1:100) = \emptyset$$

packet 1 : 1 3 5 7

packet 2 : 9 11 13 15

UC	BM UT1	BM UT2	BM UT3	BM UT4
----	-----------	-----------	-----------	-----------

load	A(1)	A(3)	A(5)	A(7)
------	------	------	------	------



do $i = 1, 100, 3$

$$C(i) = A(i) + B(i)$$

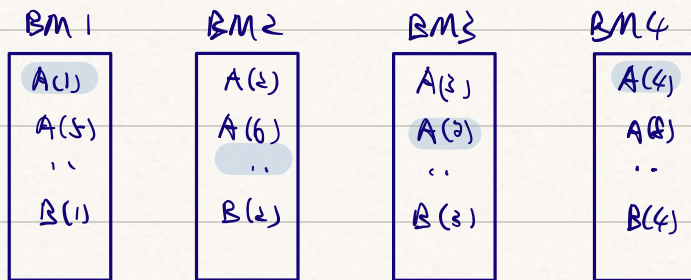
end do

$$C(1:100) \cap A(1:100) = \emptyset$$

$$C(1:100) \cap B(1:100) = \emptyset$$

UC BM
UT 1 UT 2 UT 3 UT 4

Load A(1) A(4) A(7) A(10)



$A(i) \rightarrow$ BM k t.g. $k \equiv i[4]$

probab en vrai ?

stride s

$s=1$ 0,8 + 0,1

$s=2$ 0,05

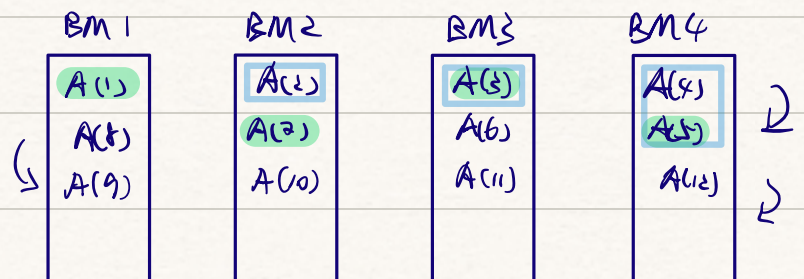
reste 0,05

probab

0,8 $2 \nmid s \Rightarrow 4$ mots / cycle

0,25 $2 \mid s$ et $4 \nmid s \Rightarrow 2$ mots / cycle

0,25 $4 \mid s \Rightarrow 1$ mot / cycle



do $i = 1, 100$

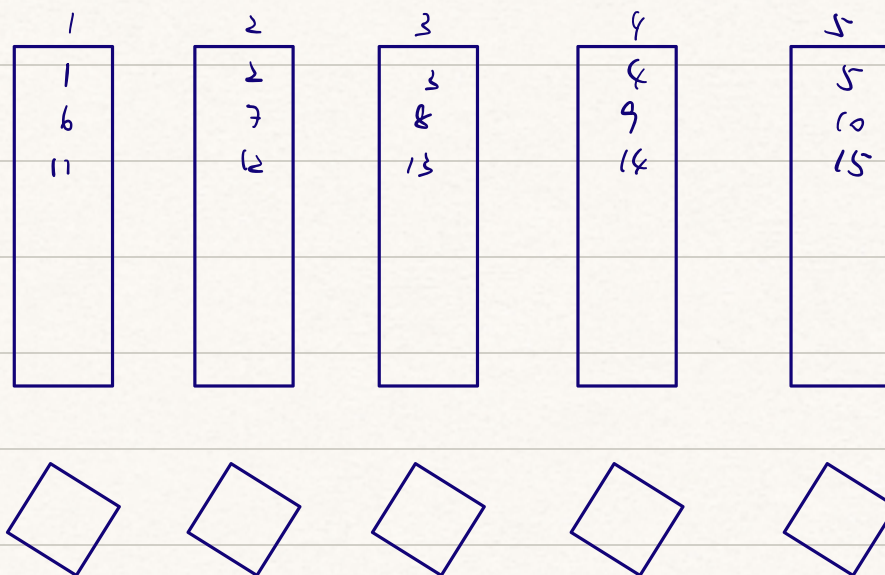
$$C(i) = A(i+1) + B(i)$$

→ problème de décalage

end do

$$C(1:100) \cap A(1:100) = \emptyset$$

$$C(1:100) \cap B(1:100) = \emptyset$$



$A(i) \rightarrow$ BM k t.q. $k \in i[5]$ ← division (base 2)
par nombre premier
⇒ compliqué

stride $0.8 \ 5 \nmid 5 \Rightarrow 5$ mots / cycle

en vrai 0.95
 $0.2 \ 5 \mid 5 \Rightarrow 1$ mot / cycle
?

2D $A(4,4)$

$A(i,j) \rightarrow$ BM $k \ k \in j[4]$

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,4
4,1	4,2	4,3	4,4

ligne OK

colonne non

BM1	BM2	BM3	BM4
1,1	1,2	1,3	1,4
2,4	2,1	2,2	2,3
3,3	3,4	3,1	3,2
4,2	4,3	4,4	4,1

colonne OK

diagonale non

ranglement ?

$A(i,j) \rightarrow \text{BM } k \text{ t.g. } i+j-1 \equiv k [4]$

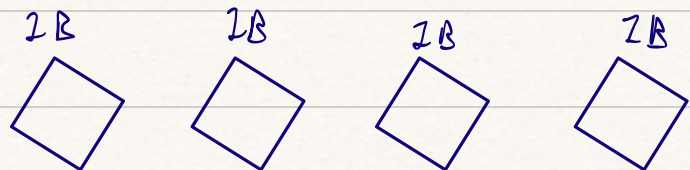
do $i \in 1, 7$

$C(i) = A(i) + B(i)$

end do

tranche 1 : 1 2 3 4

tranche 2 : 5 6 7



si $IB = 0$ NOP

si $IB = 1$ OP exécuté

UC	BM UT1	BM UT2	BM UT3	BM UT4
load	A(1) :	A(2) :	A(3) :	A(4) :

load

$$IB \geq 1 \\ B(5)$$

$$IB \geq 1 \\ B(6)$$

$$IB \geq 1 \\ B(7)$$

$$IB \geq 0 \\ \times$$

$$do \quad i = 1, n$$

$$n = 4k + r$$

$$si \quad r = 0$$

k tranches complètes

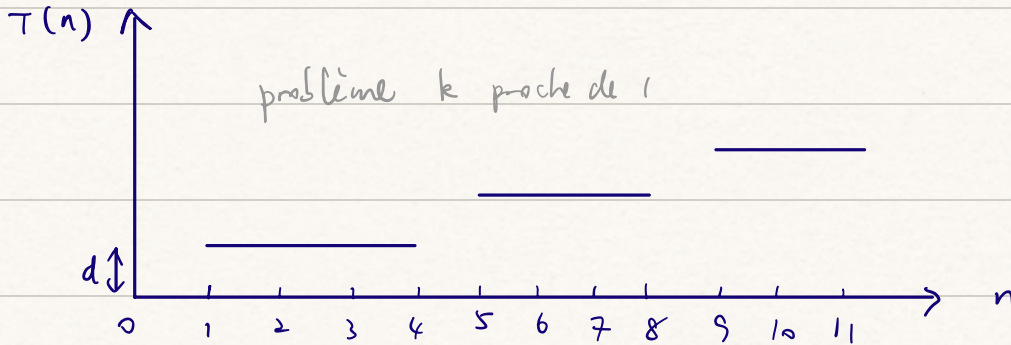
$$T(n) = k \times d$$

$$si \quad r \neq 0$$

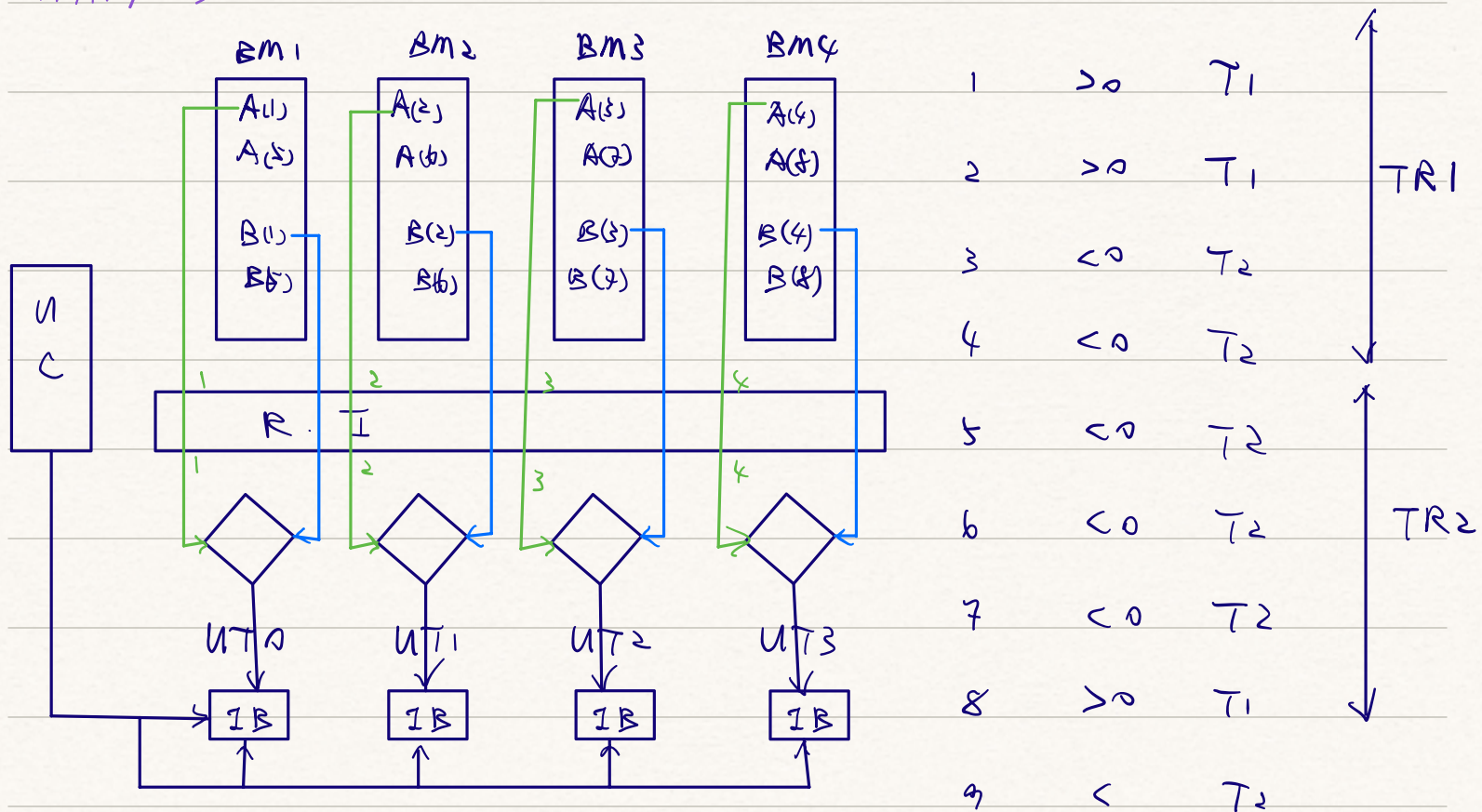
k tranches complètes

$$T(n) = (k+1) \times d$$

+ 1 tranche incomplète



17/11/2023



si IB = 0 NOP

si IB = 1 EXEC STD

do i = 1, 100

TMP₁ = F(i), TMP₂ = G(i)

if (A(i) > 0)

then T₁

else T₂

end if

end do

E(i) = F(i) * G(i)

H(i) = F(i) - G(i)

UC	UT ₁	UT ₂	UT ₃	UT ₄
load	A(1)	A(2)	A(3)	A(4)
compare	>0	>0	<0	<0
IB	1	1	0	0
T ₁	T ₁ (1)	T ₁ (2)	×	×
IB	0	0	1	1
T ₂	×	×	T ₂ (3)	T ₂ (4)
reset	1	1	1	1
load	A(5)	A(6)	A(7)	A(8)
compare	<0	<0	<0	>0
IB	0	0	0	1
T ₁	×	×	×	T ₁
IB	1	1	1	0
T ₂	T ₂	T ₂	T ₂	×

$$T_1 \quad d_1 \quad T_2 \quad d_2$$

$$\alpha \quad 1-\alpha$$

$$T_{SE}(n) = \alpha n d_1 + (1-\alpha) n d_2 \quad \text{temps séquentiel}$$

$$n \geq 4k$$

$$T_{SI}(n) = k(d_1 + d_2) = \frac{n}{4}(d_1 + d_2)$$

$$\alpha = 0,5$$

$$T_{SE}(n) = n \left(\frac{d_1 + d_2}{2} \right)$$

$$\frac{T_{SE}(n)}{T_{SI}(n)} = \frac{\frac{n}{2}(d_1 + d_2)}{\frac{n}{4}(d_1 + d_2)} = 2$$

9. 快 2 倍 $\rightarrow \frac{1}{2}$ 冗余

do $i \geq 1, 100$

$C(i) \geq A(i) + B(i)$

end do

do $i \geq 1, 100$

$C(i) \geq A(i) + B(i+1)$

end do

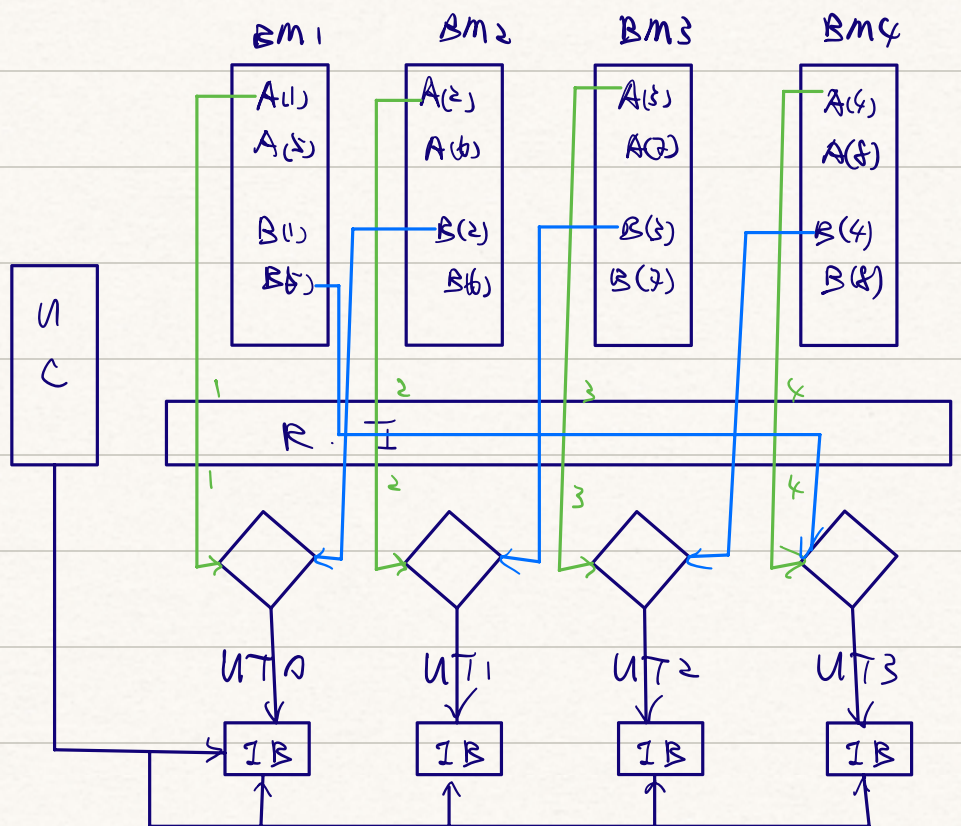
in out

1 4

2 1

3 2

4 3



$i \rightarrow j$ i.g. $j \equiv i+3 [4]$

range ment de données cyclique

$\Rightarrow$ décalages circulaires sur le réseau

réseau $k \times k$ décal. d

$\Rightarrow$ in $i \rightarrow$ out $j \equiv i+d[k]$

avantages

hardware simple

1 UC

programmation simple

désavantages

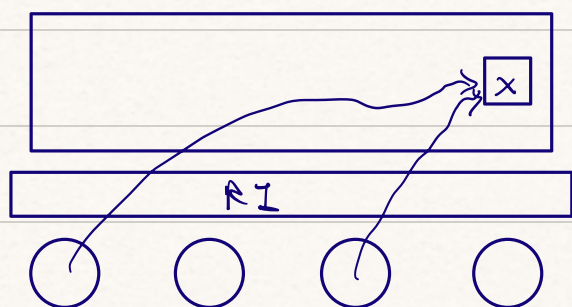
codes réguliers et uniformes

graphique : GPU

10^3

IA

shared memory



P1

P3

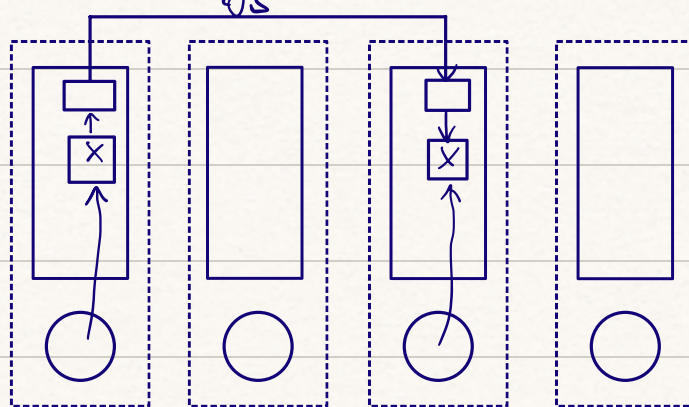
processus

x

x

$\leftarrow wait(x, p_1)$

distributed memory



x
 $send(x, p_3)$

$receive(x, p_1)$
 x

OS runtime

~ milles cycles

do $i = 1, 100$

$C(i) = A(i) + B(i)$

L1

end do

L2

do $i = 1, 40$

$D(i) = C(i)$

end do

$C(1:100) \cap A(1:100) = \emptyset$

$C(1:100) \cap B(1:100) = \emptyset$

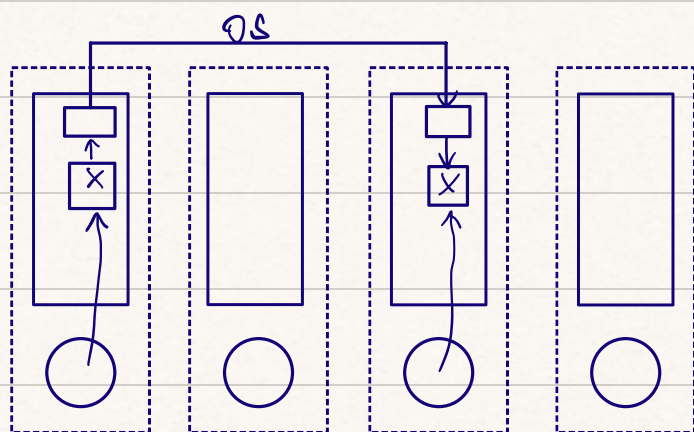
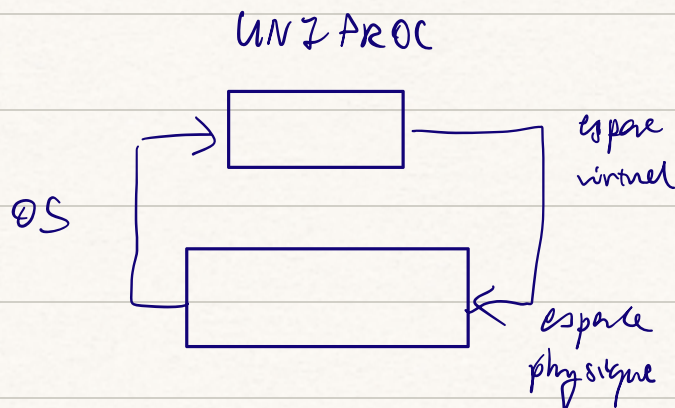
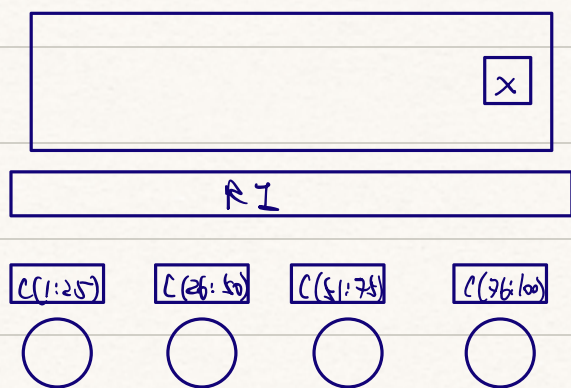
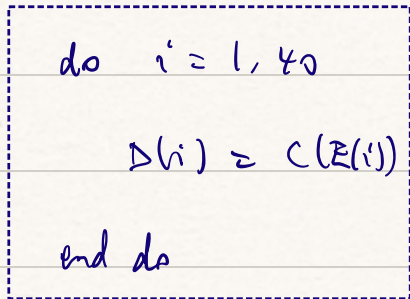
C1	C2	C3	C4	
1 → 25	26 → 50	51 → 75	76 → 100	L1
1 → 10	11 → 20	21 → 30	31 → 40	L2

C1	C2	C3	C4	
1 → 25	26 → 50	51 → 75	76 → 100	L1
A (1 → 25) B C	A (26 → 50) B C	A (51 → 75) B C	A (76 → 100) B C	

1 → 10	11 → 20	21 → 30	31 → 40	L2
C (1 : 10)	C (11 : 20)	C (21 : 30)	C (31 : 40)	
send C2	receive C1	receive C1	receive C2	
C (11 : 20)	C (11 : 20)	C (21 : 25)	C (31 : 40)	
send C3	send C3	receive C2		
C (21 : 25)	C (26 : 30)	C (26 : 30)		
	send C1			
	C (31 : 40)			

receive	receive			L3
:	:	:	:	

L3



C1



send / receive

Message

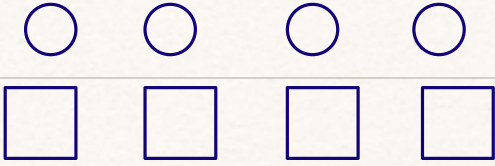
Passing

Interface

shared virtual memory

open MP

g ner espace partag 



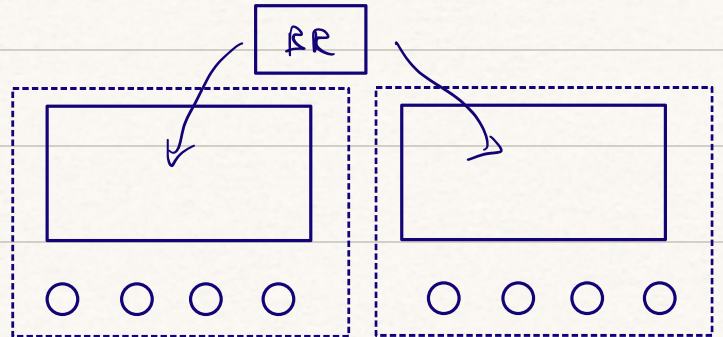
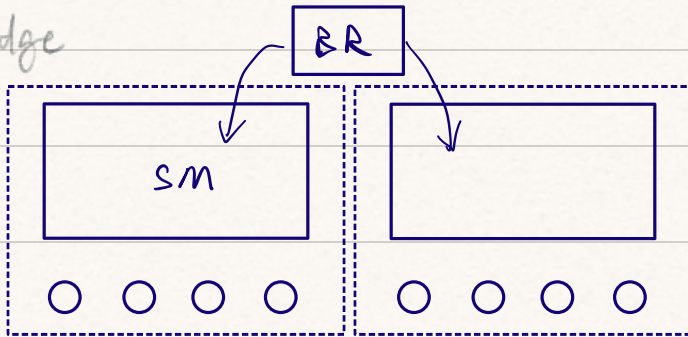
Non

Uniform

Memory

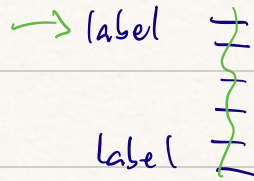
Access

bridge



24/11/2023

basic block

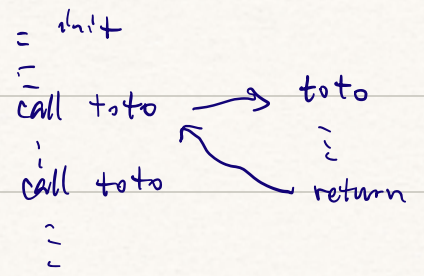


séquentiel

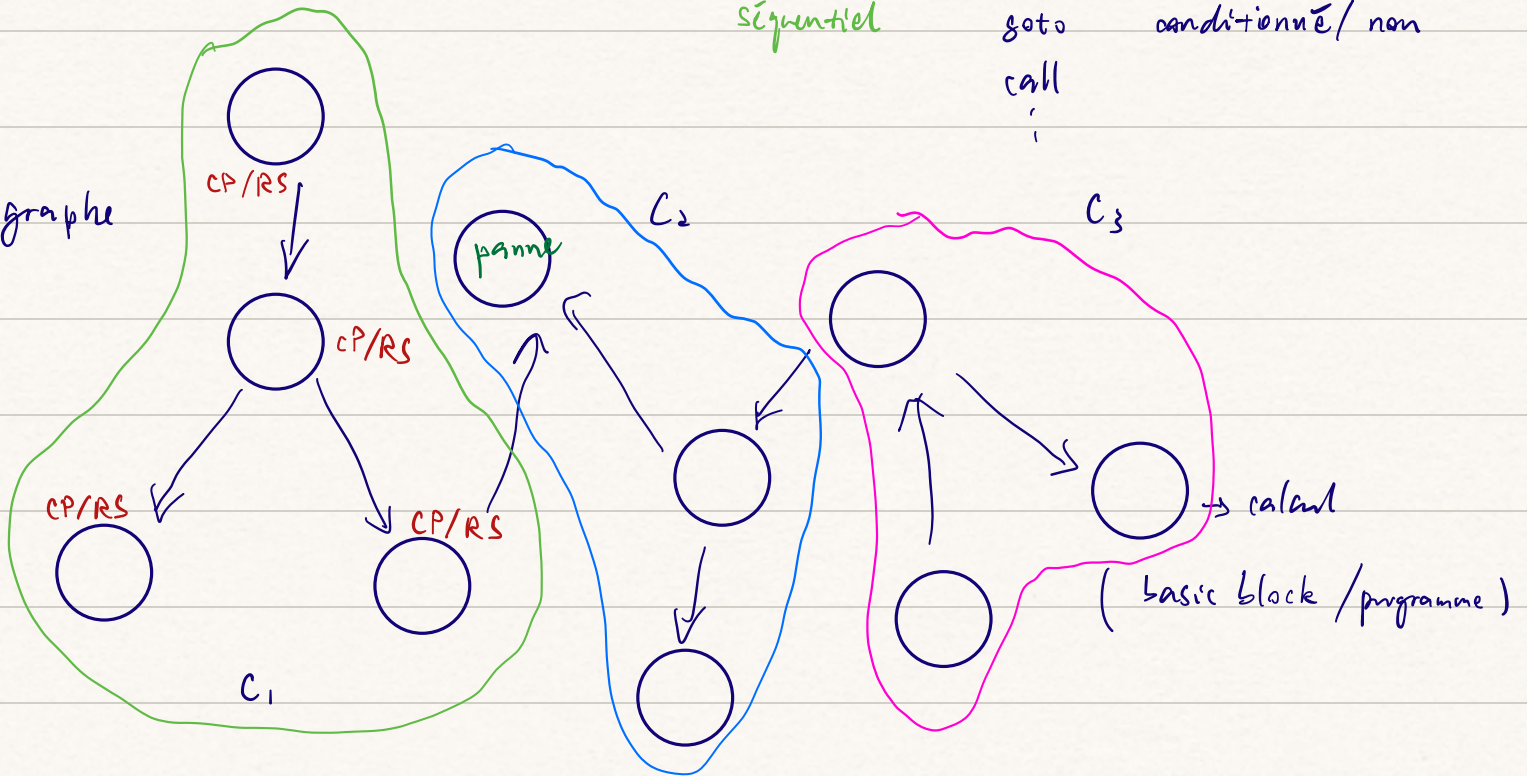
branch

if
goto
call
;

conditionné / non



graphe



noeuds : commun data / synchronisation

cible : 3 cœurs

parallélisation $\Leftrightarrow$ partitionnement

C_1, C_2, C_3 numéros virtuels

1 noeud $\rightarrow$ associé à un seul cœur

3 cœurs parmi 16 cœurs

OS attribue les cœurs $CP_1 \dots CP_{16}$

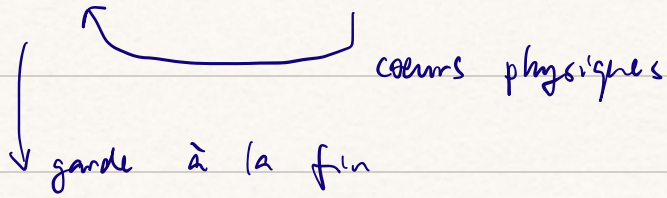
(cœur physique)

$C_1 \rightarrow CP5$

$C_2 \rightarrow CP8$

$C_3 \rightarrow CP12$

programme $\rightarrow$ OS (nombre de cœurs)



GANG scheduling

pannes C_2 ont

(solution possible : redistribution de cœur, re-commence de zéro)

à fin de chaque tâche sauve garde résultat

CP/RS check point restart $\rightarrow$ user

① partitionnement statique / explicite / user $\rightarrow$ parametre

② scheduling OS / dynamique

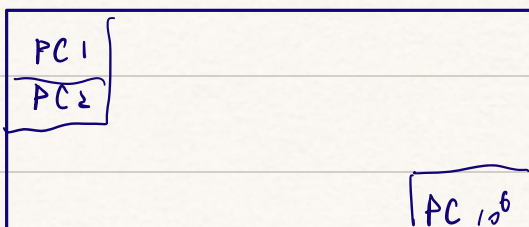
mapping sous graphes virtuels $\rightarrow$ cœurs physiques

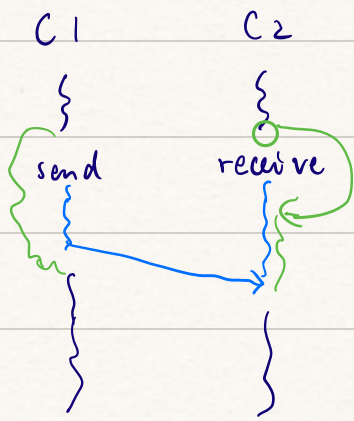
③ communication mémoire distribuée

④ synchronisation mémoire partagée $send()$, $receive()$

⑤ équilibrage de charge

10^6 cœurs





$$C_1: T_{cal1} + T_{com1} + T_{w1}$$

$$C_2: T_{cal2} + T_{com2} + T_{w2}$$

prog spécialisée communication

$$C_1 \approx \max(T_{cal1} + T_{com1})$$

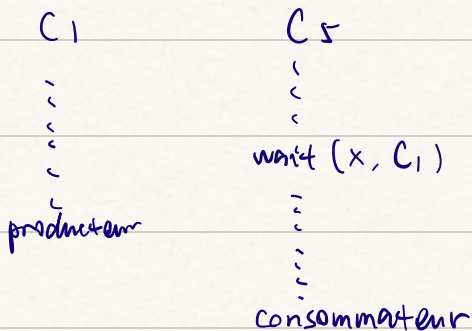
$$T_{cal} + T_{com} \Leftarrow \max(T_{cal} + T_{com})$$

gain: $S = \frac{T_{cal} + T_{com}}{\max(T_{cal}, T_{com})} \leq 2$

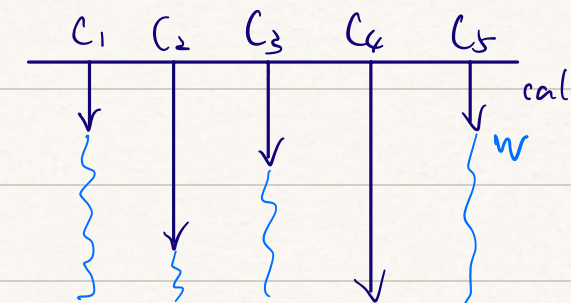
(parallélisation calcul, communication)

→ limiter T_{com}

synchronisation 3 types



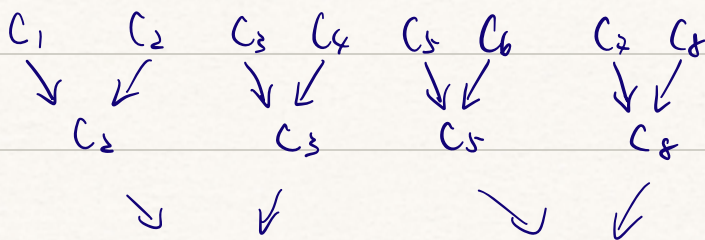
barrière



tous les cœurs s'attendent

→ disponibilité

→ coût



do $i = 1, 1000$

calcul $A(i)$ en parallèle

$s = s + A(i)$

end do

2 coeurs

$s = 10$

$A(1) = 20$

$A(2) = 30$

$\rightarrow s = 60$



do $i = 1, 2$

calcul $A(i)$ 1 seul cœur

begin critical section

$s = s + A(i)$

end critical section

end do

best

T_1 : temps exécution dans 1 cœur

T_p temps exécution sur p cœurs

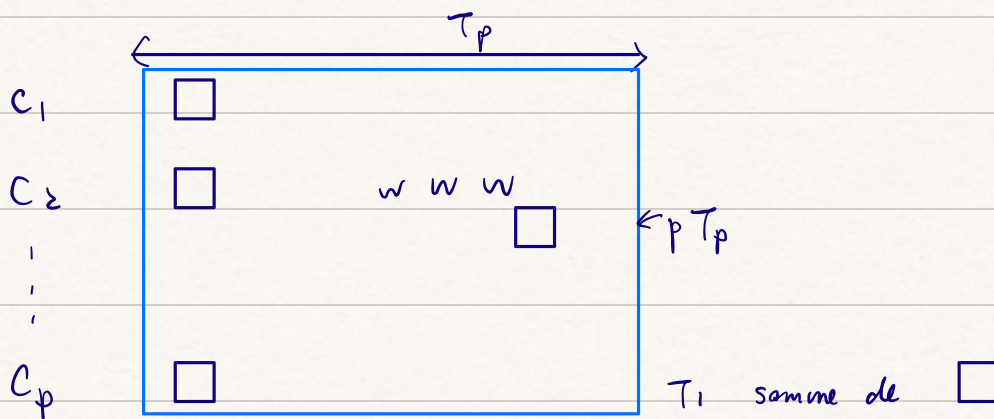
speedup :

$$S_p = \frac{T_1}{T_p}$$

$$S_p \leq p$$

efficacité

$$E_p = \frac{T_1}{p T_p} = \frac{S_p}{p}$$



$\square$: C, S, W, Com

ex. anomalie

mémoire

2M

2M

2M

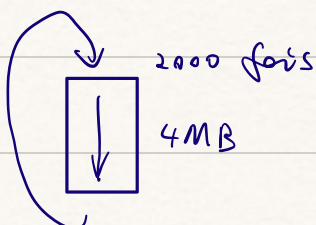
2M

(C)

(C)

(C)

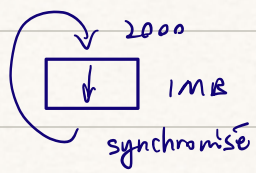
(C)



$$T_1 = 2000 \cdot \frac{4MB}{V_m}$$

V_m : vitesse read mem

(1e lecture)



$$T_4 = 2000 \cdot \frac{1MB}{V_c}$$

V_c : vitesse read cache

$$V_c \geq 10 V_m$$

$$\frac{T_1}{T_4} = 4 \frac{V_c}{V_m} \geq 40 \rightarrow T_1 \text{ pas meilleur}$$

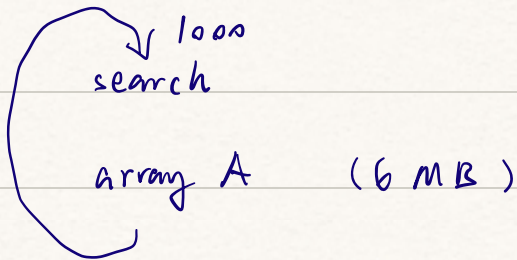
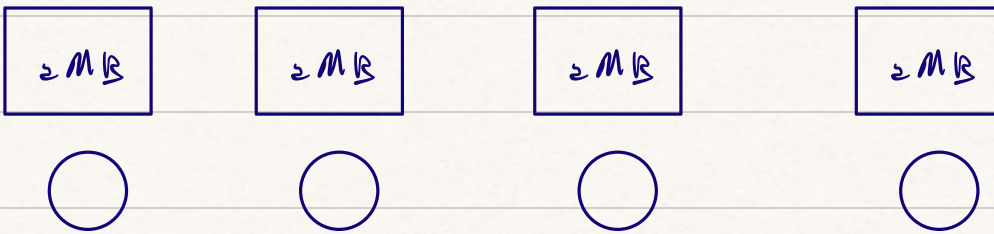
01/12/2023

$$\frac{T_1}{T_p} \approx p$$

T_1 : temps exécution sur 1 processeur
 T_p

W: want

1: idle



$$T_1 = (\text{read } A) * 1000 \approx \frac{6MB}{V_m} \quad V_m: \text{vitesse mémoire}$$

do $i = 1 : 100$

if $(A(i) > 0)$

then T_1 100 单位时间

else T_2 1

end if

end do

$A(1:25) > 0$

↓ T_1

$A(26:100) < 0$

↓ T_2

mémoire partagée 4 cœurs

static scheduling

C_1

C_2

C_3

C_4

1-25

26-50

51-75

76-100

T_s

T_s

T_s

T_s

(synchronisation)

$25 * 100$

$25 * 1$

$25 * 1$

$25 * 1$

→ récupérer ce qu'on a fait

$2500 + T_s$

$25 + T_s$

$25 + T_s$

$25 + T_s$

$= T_{EXEC}$

2501 $T_s = 1$

2700 $T_s = 200$

dynamic scheduling : chaque cœur cherche 1 itération puis calcule
puis reprend

C_1	C_2	C_3	C_4
T_s	T_s	T_s	T_s
100	100	100	100
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$600 + 6T_s$	$600 + 6T_s$	$600 + 6T_s$	$600 + 6T_s$
$100 + T_s$ ($\leftarrow \text{rest}$)	$1 + T_s$	$1 + T_s$	$1 + T_s$
$\downarrow$			
$700 + T_s$			
	75	$T_s = 1$	
		$\downarrow$ per 3 columns	
707	$T_s = 1$	$606 + 50$	$606 + 50$
2100	$T_s = 200$	$1800 + 201$	$1800 + 201$
+201		+201	
		=2202	
~ 3600			

block scheduling :

packets:

$P_1 : 1-4$	400
$P_2 : 5-8$	400
$\vdots$	$\vdots$
$P_6 : 21-24$	400
$P_7 : 25-28$	400
$\vdots$	$\vdots$
$P_{25} : 96-100$	4

C1	C2	C3	C4
Ts	Ts	Ts	Ts
P1	P2	P3	P4
Ts	Ts	Ts	Ts
P5	P6	P7	P8
,	- - -		
:			

$2 \times 400 + 2T_s$		$503 + 2T_s$	$404 + 2T_s$
$T_s = 1$ 802	802	505	406 + 5 * 12

$T_s = 200$ 1200	1200	903	804
+ 204 * 4 ⁸¹⁶	+ 204 * 4	+ 204 * 4	+ 204 * 5 1020

cache : case mémoire consécutives

N itération P colonnes $N = P \cdot K$

	static	dynamic	block
Ts	$P \cdot T_s$	$N \cdot T_s$	$\frac{N}{\text{block size}} \cdot T_s$
LB load	- -	+ +	+
localisation spatiel	+ +	- -	bloc
	↑ pas de "if"		↑ "if" varié

dynamique

loop scheduling :

static

distribution équilibrée

dynamique

block

$P \times O(P) T_s$

GSS

guided self scheduling

distribution
inconnue

(nb de synchronisation faible) bloc taille variable

100 iteration 4 cœurs

$$B_1 = \left\lceil \frac{100}{4} \right\rceil = 25$$

$$B_2 = \left\lceil \frac{75}{4} \right\rceil = 19$$

$$B_3 = \left\lceil \frac{56}{4} \right\rceil = 14$$

$$B_4 = \left\lceil \frac{42}{4} \right\rceil = 11$$

$$B_5 = \left\lceil \frac{31}{4} \right\rceil = 8$$

$$B_6 = \left\lceil \frac{23}{4} \right\rceil = 6$$

$$B_7 = \left\lceil \frac{17}{4} \right\rceil = 5$$

$$B_8 = \left\lceil \frac{12}{4} \right\rceil = 3$$

$$B_9 = \left\lceil \frac{9}{4} \right\rceil = 3$$

$$B_{10} = \left\lceil \frac{6}{4} \right\rceil = 2$$

$$B_{11} = \left\lceil \frac{4}{4} \right\rceil = 1$$

$$B_{12} = \left\lceil \frac{3}{4} \right\rceil = 1$$

$$B_{13} = \left\lceil \frac{2}{4} \right\rceil = 1$$

$$B_{14} = \left\lceil \frac{1}{4} \right\rceil = 1$$

(manipule localisée en fin)

→ limite la taille bloc
(eg. min = 2)

$$\sum T_{EXEC} = \sum T_{comp} + \boxed{\sum T_{synch/w} + \sum T_{comm}}$$

over head

objectif :

$$\frac{\sum T_{synch/w}}{\sum T_{EXEC}} \leq 10\% \quad \text{et} \quad \frac{\sum T_{comm}}{\sum T_{EXEC}} \leq 10\%$$

08/12/2023

réseaux

→ généralités

→ grid 2D / 3D / torus

→ hypercube

→ xbar

→ réseau Σ

→ SIMD data unscrambling

réorganisations de données

tranche / word

synchrone

→ MIMD mémoire partagée

moyen d'accès en parallèle mémoire

cache line

asynchrone

→ MIMD mémoire distribuée

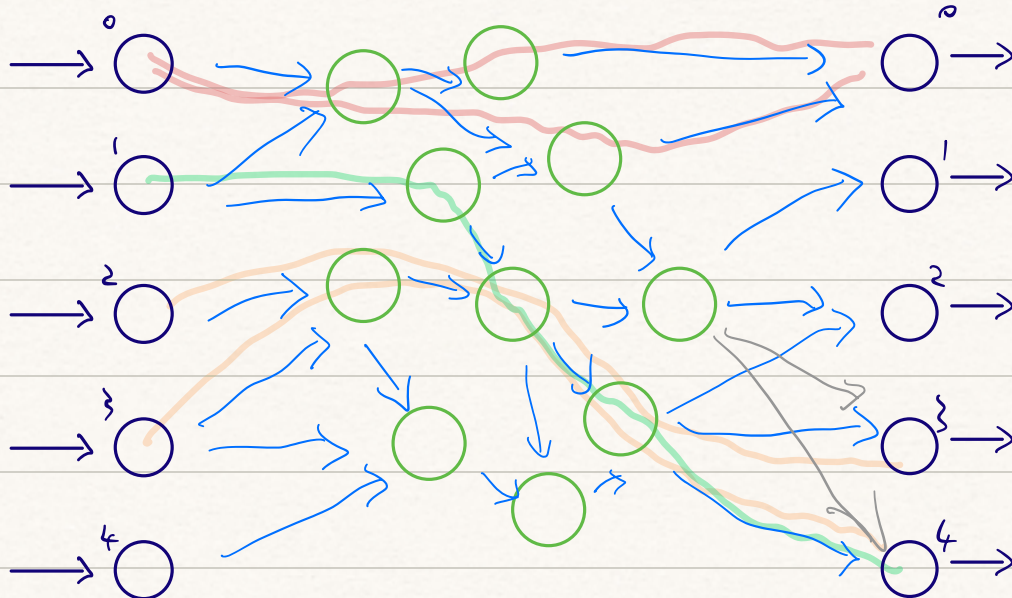
échange de messages

page 4KB

asynchrone

in

out



croisement :
couches différentes
→ congestion

proc

routeurs

mem



liens nm / bi-directionnel ~ cable : width nb, bits

chemin : ensemble nœuds + liens

at least 1 path ✓

more than 1 ?

cast : • nb. ports

• mem. size

• layout

• length

• nb.

conflicts : $\rightarrow$ liens

$\rightarrow$ ports

détection

résolution : mise en attente



coût stockage

in $x \rightarrow$ out y

algo routage

table (variantes)

$d(x, y)$: nb. min. routeurs

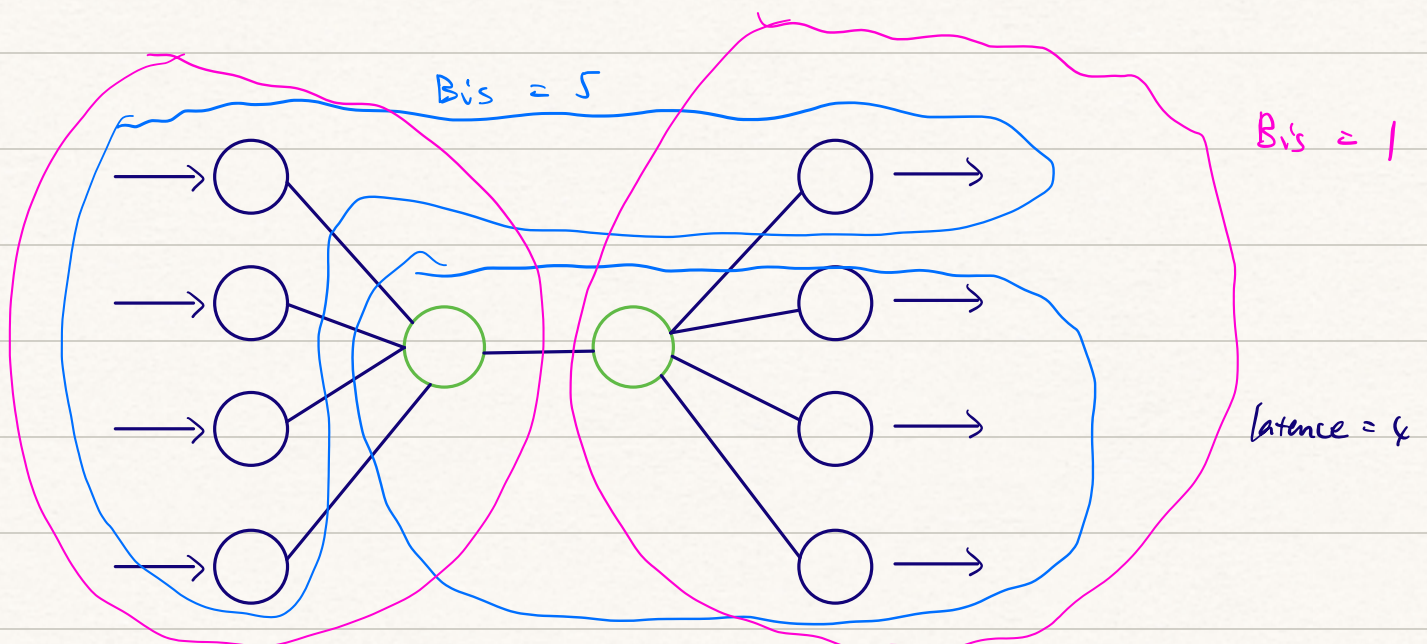
algo routage optimal

$\forall x, y, d(x, y)$ routeurs

latence = $\max_{\substack{x \in \text{in} \\ y \in \text{out}}} d(x, y)$

$$d(0, 0) = 4$$

$$d(0, 4) = 5$$



N nœuds $\rightarrow$ partition $\mathcal{P}$ en 2 sous ensembles de $\frac{N}{2}$ nœuds
($\mathcal{P}_1, \mathcal{P}_2$)

nb. liens entre $\mathcal{P}_1, \mathcal{P}_2 = \text{Bis}(\mathcal{P}_1, \mathcal{P}_2)$ (bisection)

$\min_{\mathcal{P}_1, \mathcal{P}_2} \text{Bis}(\mathcal{P}_1, \mathcal{P}_2)$ (point faible de communication)

SIMD : permutation décalage circulaire

1024 in $\rightarrow$ 1024 out

$i \rightarrow j \equiv i+12 [1024]$ 10 bits

source, destination

MIMD 1024 (S, D) $S \in [0, 1023]$ $D \in [0, 1023]$

1024 * (10 + 10) 10 Kbit

réseau : graphe

latence : chemin critique

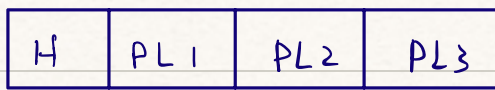


in order

destination : out
source : in
bits de parité (sécurité) $\rightarrow$ message à peu près bien déroulé

size(H) / size(payload) = (10 + 10 + 8) / 64 B = 28 / 512
machine 1024 * 1024 " 512 bits

②

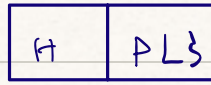
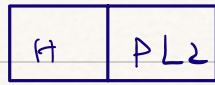


queue LEU

in order



③

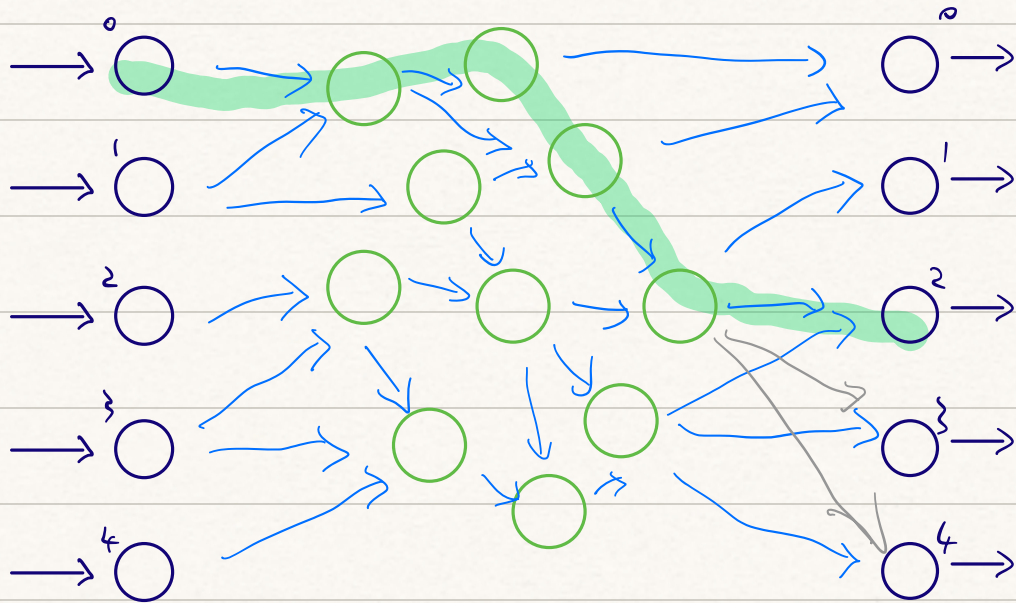


(header triplé : pas malin)

out of order

circuit switching

→ header établit / réserve le chemin



→ message (PL) envoyé

→ pas de stockage

→ transfert ultra rapide

→ mauvaise utilisation réseau

→ long à établir

→ blocages nombreux

store and forward

→ déplacement par bond



→ stockage interne dans chaque nœud

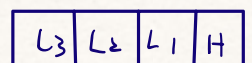
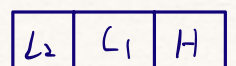
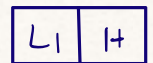
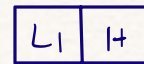
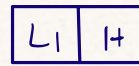
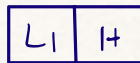
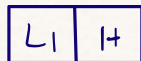
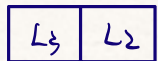
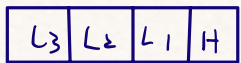
① ③

→ vision : $\geq$ nœuds + 1 lien

→ "bonne" utilisation réseau
minimale

→ peu de blocages

worm hole routine ②

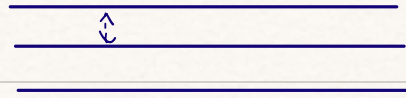


stockage nœud

"pipeline"

15/12/2023

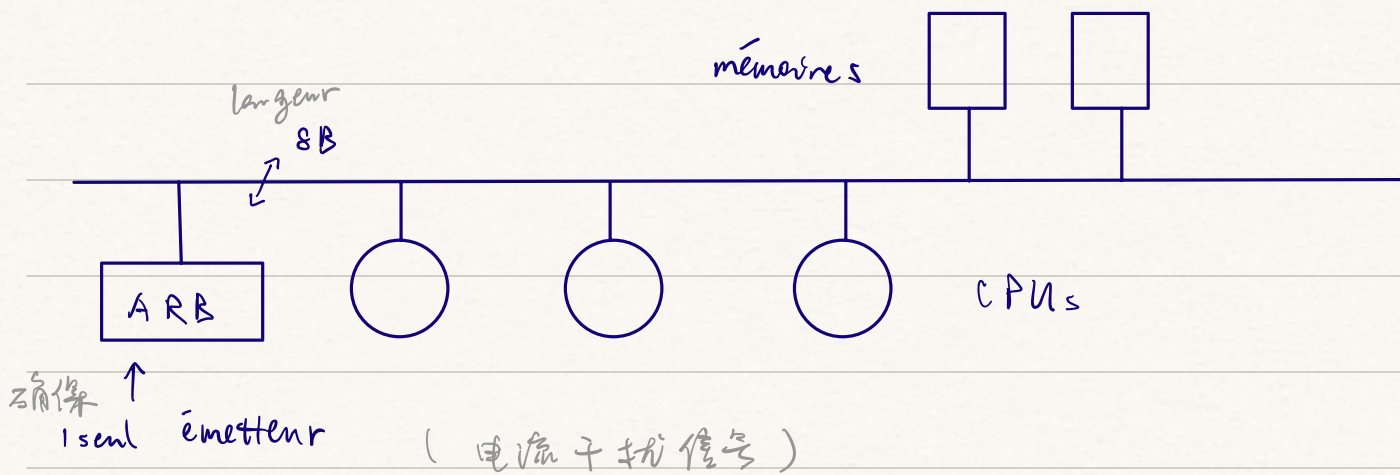
bus



"cross talk" diaphone

$$\text{bandwidth} = \text{freq} \times \text{width} \quad (\text{PCB})$$

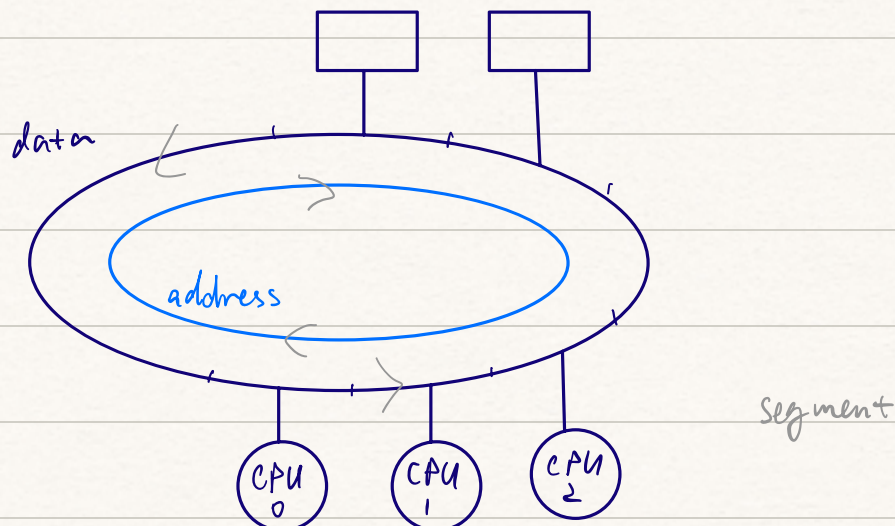
$$\text{board} \quad 1 \text{ GHz} \times 8 \text{ B} = 8 \text{ GB/s} \quad (\text{DIE})$$



$$\text{arbitrer} : (\text{nb. CPU} + \text{nb. mem}) \times \text{freq.} = \text{nb. requête / sec.}$$

bus : < 16 émetteurs

pipe line

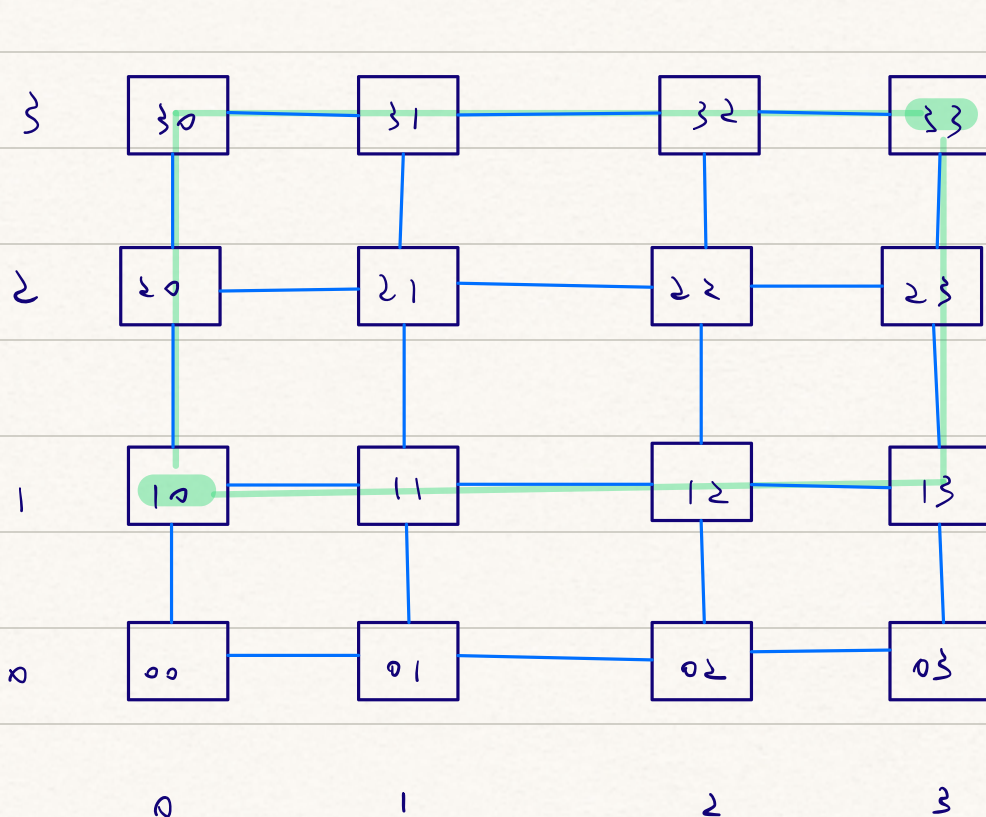
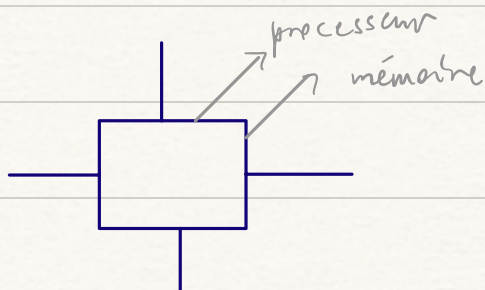


pas d'arbitrer

bus : avantage centralisation

limitation: nb. émetteurs

routeur:



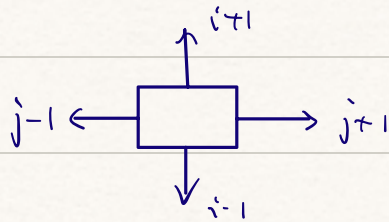
(i', j')
 $(i_s, j_s) \Rightarrow (i_d, j_d)$
 $(1,0) \rightarrow (3,3)$
 $N/S \rightarrow (3,0) \rightarrow E/S$
 optimal

	grid 4x4	grid N x N	torus 4x4	torus N x N
routeurs	16	N^2	16	N^2
liens	24	$2N(N-1)$	32	$2N^2$
liens à chaque degré routeur	≤ 4	≤ 4	4	4
latence	7	$2N-1$	5	$\frac{N}{2} + \frac{N}{2} + 1 = N+1$
bisectional bandwidth	4	N	8	$2N$
couche métalle	1	1	3	3

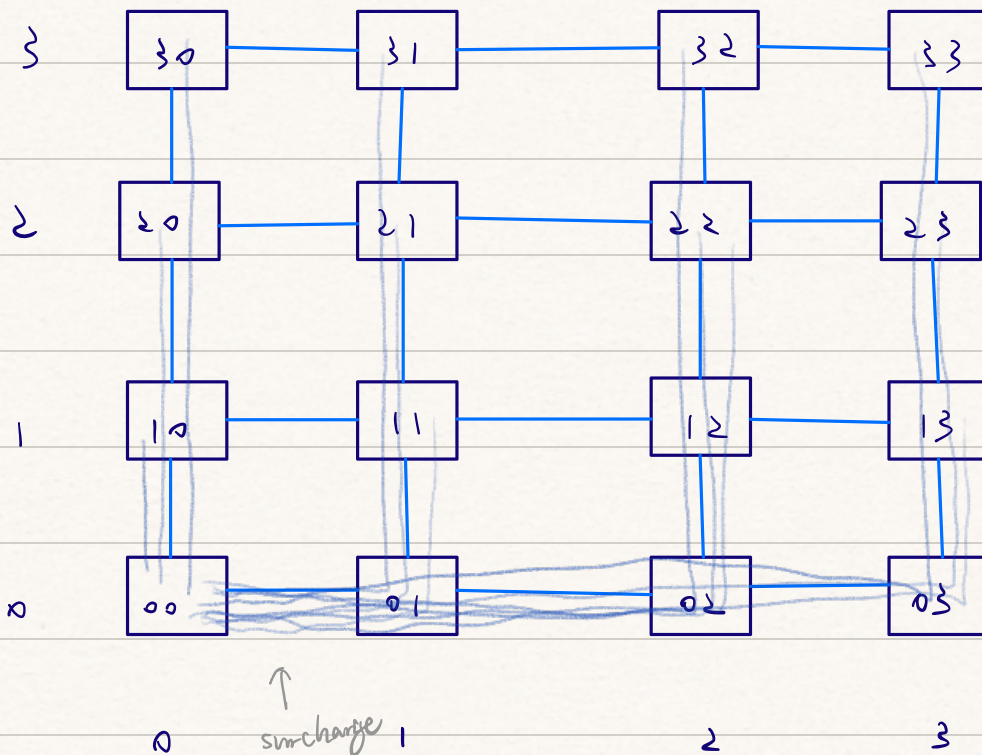
distance Manhattan

$$D((i_s, j_s), (i_D, j_D)) = |i_s - i_D| + |j_s - j_D|$$

montage optimal : à chaque étape, $D - 1$



$$\text{distance}(S, D) = D + 1$$

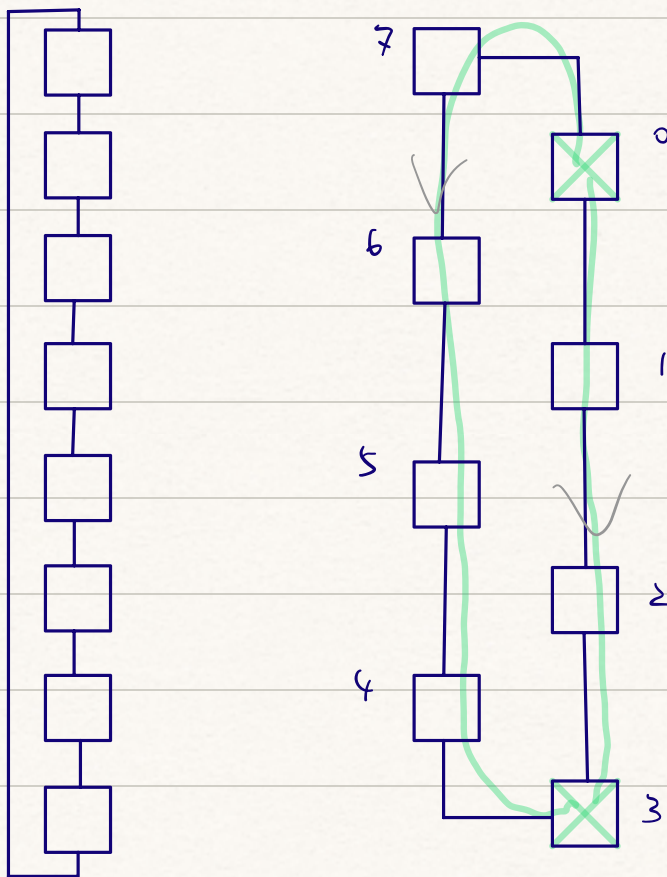
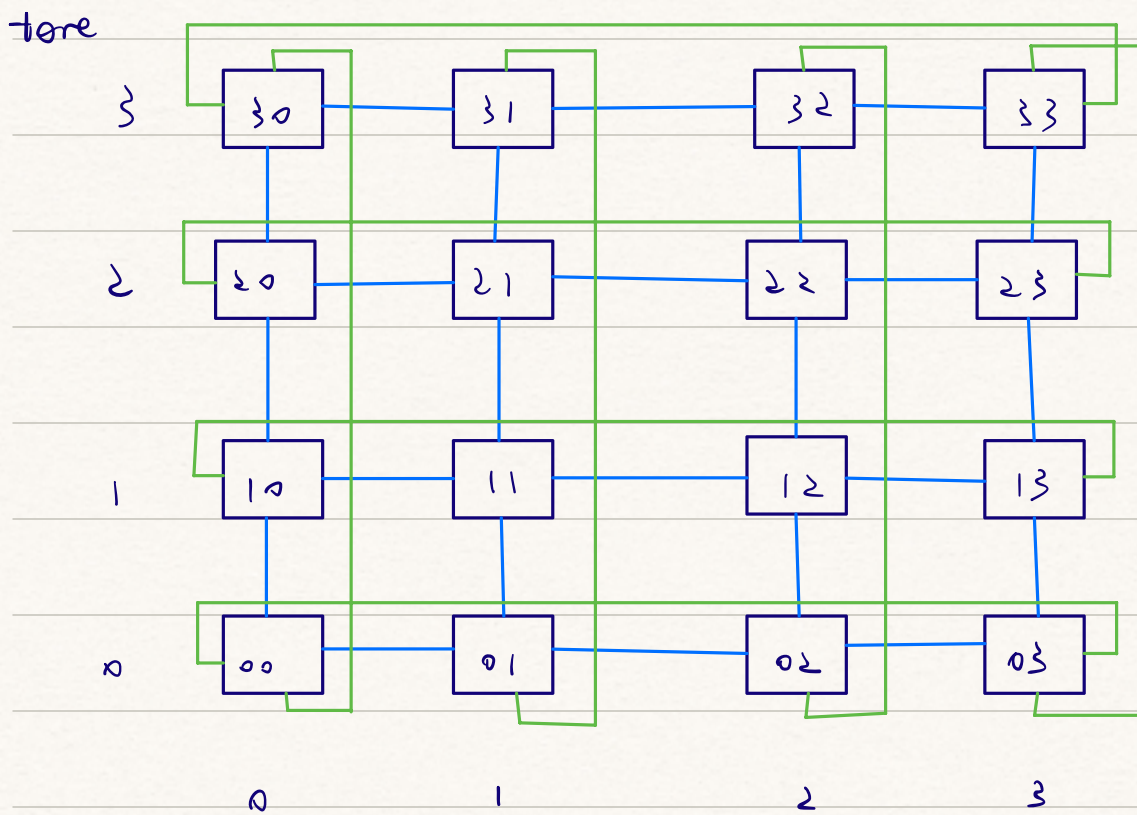


N/S puis E/O $\forall (i, j) \rightarrow (0, 0)$

$(i_s, j_s) \rightarrow (i_D, j_D)$

$\rightarrow (i_R, j_R) \rightarrow$

$\frac{1}{2} \text{ N/S} , \frac{1}{2} \text{ E/O}$



$$(0-3) \equiv 3 [8]$$

$$(0-7) \equiv 1 [8]$$

distance Manhattan torre $N \times N$

$$D(S, D) = (i_s - i_D) [N] + (j_s - j_D) [N]$$

3D grid N^3

routeurs	N^3
liens	$3N(N^2 - N - 1)$
degré	≤ 6
latence	$3N - 1$
bisectional bandwidth	N^2
couche métacle	$\sim N$

2D grid 64×64

3D grid $16 \times 16 \times 16$

torus 64×64

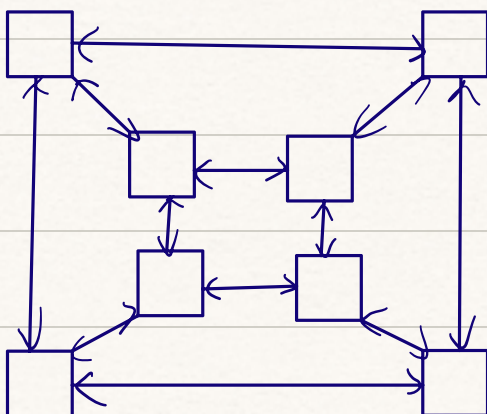
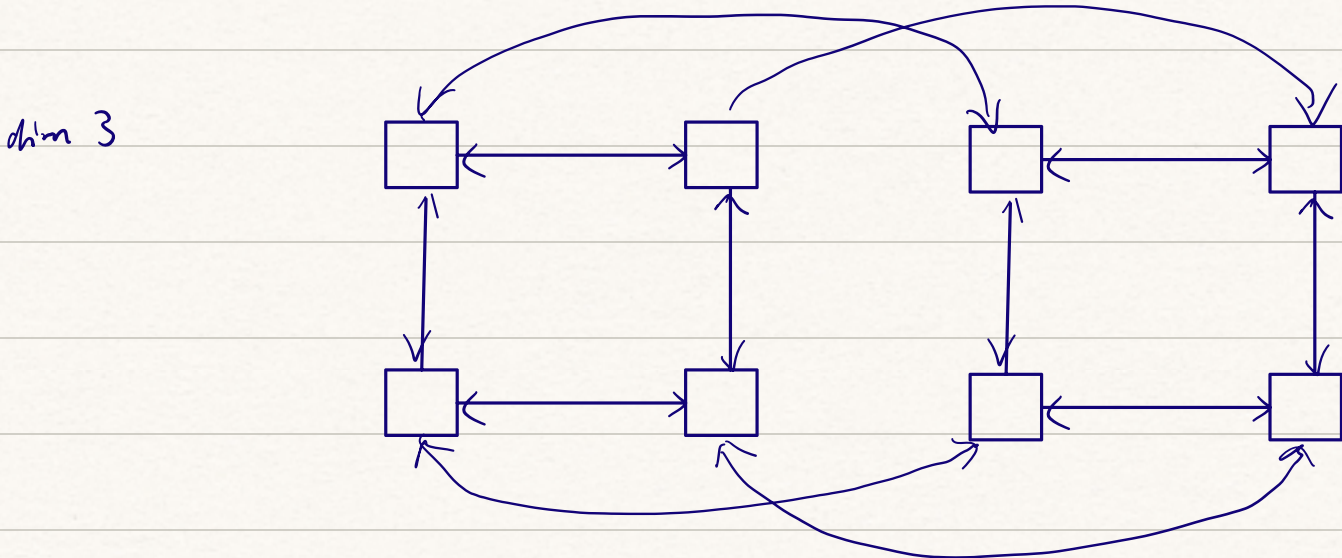
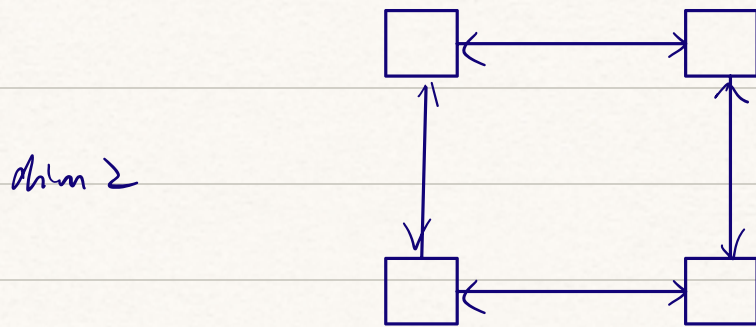
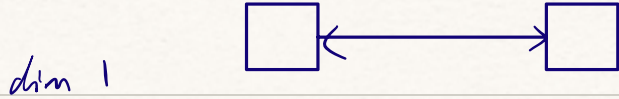
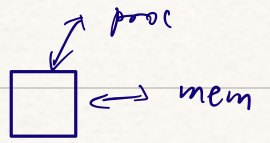
routeurs	4096	4096	4096
liens	128 x 63	3 x 256 x 15	2 x 4096
degré	4	6 1.5x	4
latence	127	47 3x	65
bisectional bandwidth	64	256 4x	128
couche métacle	1	16	3

18/12/2023

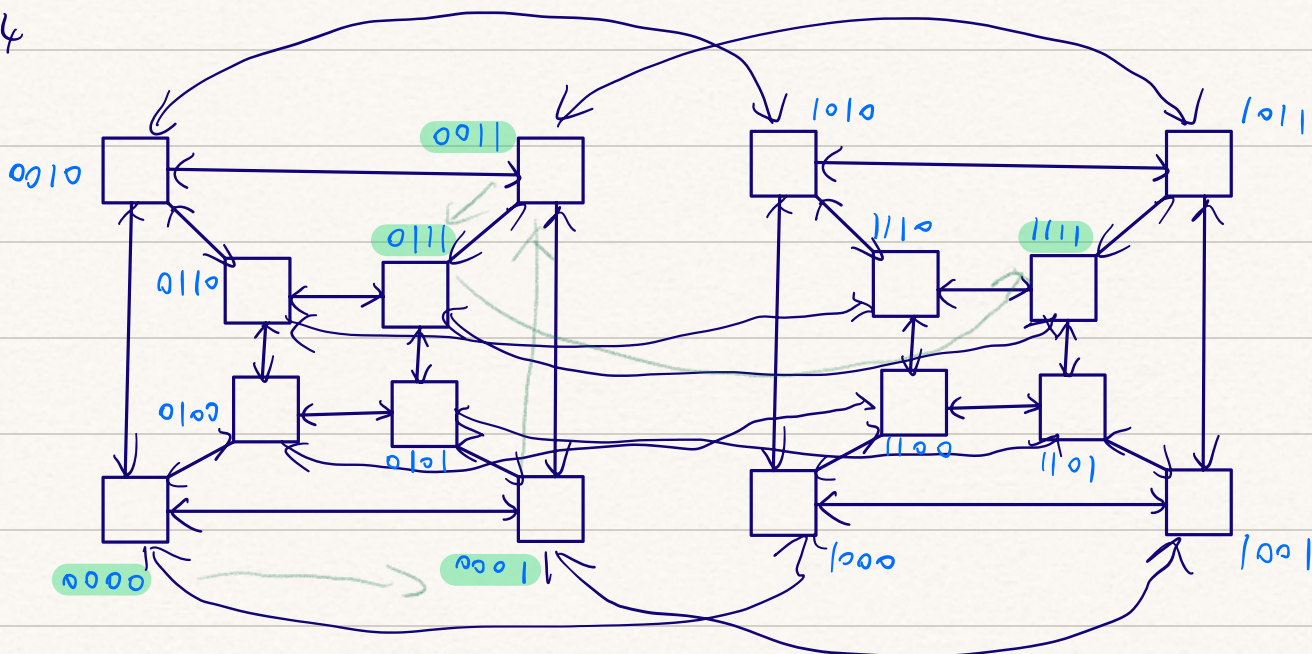
grid $N \times m$

hyper-cube 2^k k dimension

recursive



dim 4



$(\mathbb{Z}/2\mathbb{Z})^k$ espace

$\begin{matrix} \curvearrowright & \rightarrow & \uparrow & \rightarrow \\ x & x & x & x \end{matrix}$

$i_3 \ i_2 \ i_1 \ i_0$

$j_3 \ j_2 \ j_1 \ j_0$

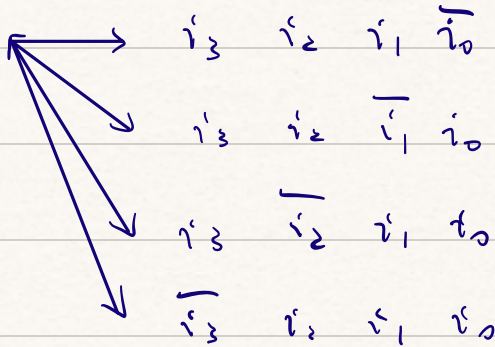


direct link

$\Leftrightarrow$

3 bits identiques

$i_3 \ i_2 \ i_1 \ i_0$



avantage

optimale

$s_3 \ s_2 \ s_1 \ s_0$

$\rightarrow$

$d_3 \ d_2 \ d_1 \ d_0$

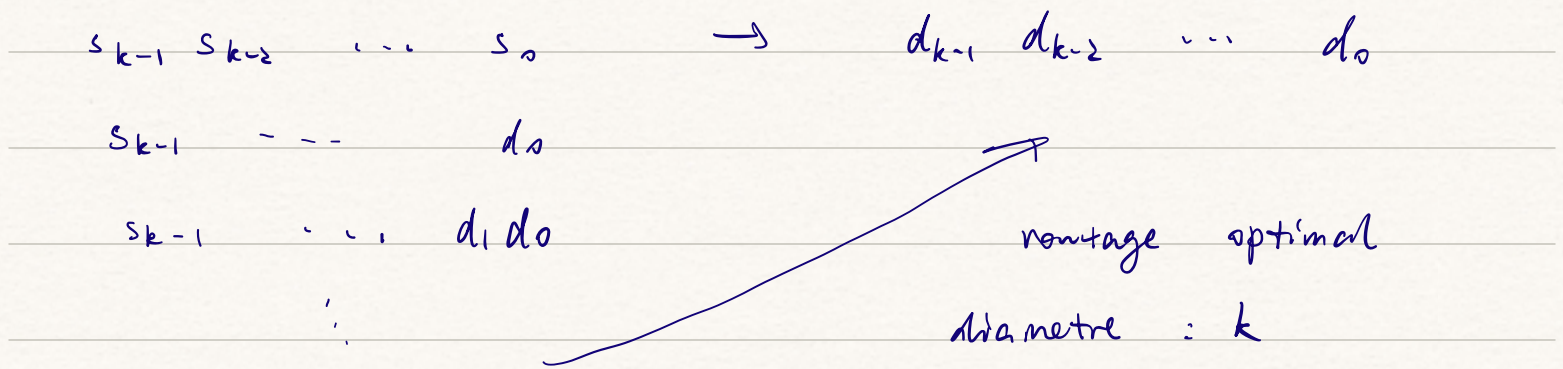
$s_3 \ s_2 \ s_1 \ d_0$

$s_3 \ d_2 \ d_1 \ d_0$

$s_3 \ s_2 \ d_1 \ d_0$

diamètre : 4

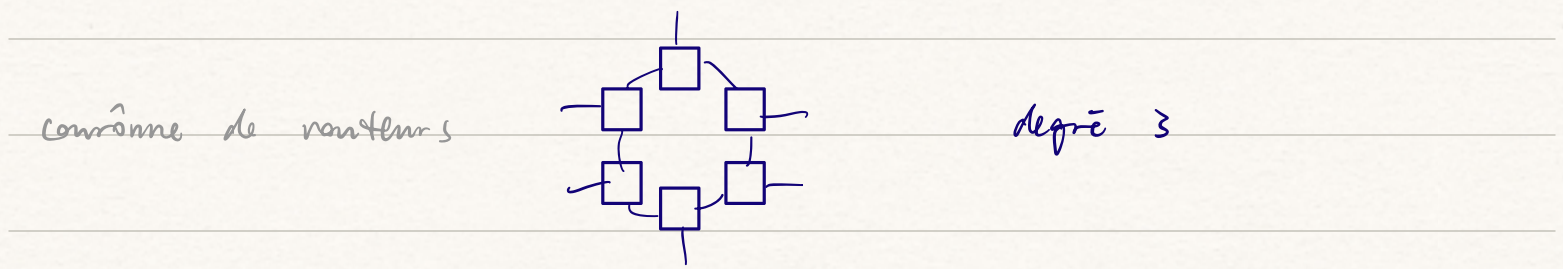
(最长路径)



$k!$ routages optimaux (changer ordres)

	hypercube 2^4	hypercube 2^k	hypercube 2^{12}	2D grid 64^2	3D grid 16^3
routeurs	16	2^k	4096	4096	4096
liens	$\frac{4 \times 16}{2} = 2^5$	$k \cdot 2^{k-1}$	$12 \cdot 2^{11}$ 2x	128x63	$3 \times 256 \times 15$
degré	4	k	12 2x	≤ 4	≤ 6
latence	4	k	12 4x	127	47
bisectional bandwidth	8	2^{k-1}	2048 8x	64	256
couche métaréseau	3	$k-1$	11	1	16

bits
 64×12 câbles



Cube numérotation $(i, j_{k-1} \dots j_0)$ $i \in [0, k-1]$

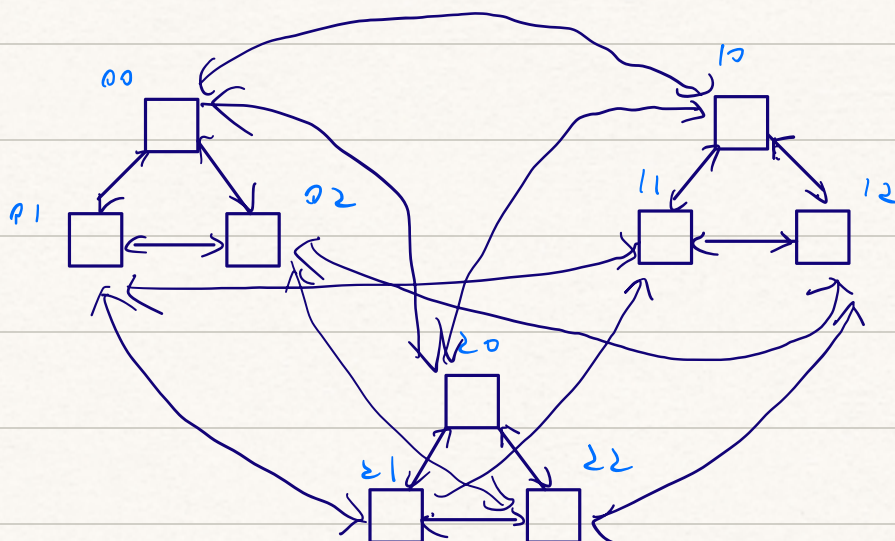
Connected

Cycle

$(\mathbb{Z}/3\mathbb{Z})^k \quad (j'_{k-1}, \dots, j'_0) \quad j'_r \in \{0, 1, 2\}$

$(j'_{k-1}, \dots, j'_0)$
 $(i'_{k-1}, \dots, i'_0)$ } direct link $\Leftrightarrow j'-1$ composantes identiques

→ généralisation hypercube

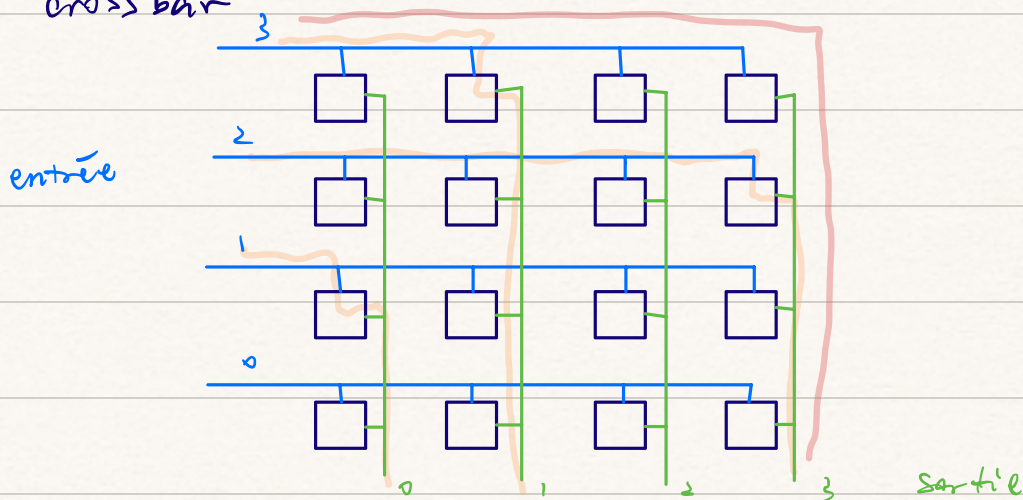


chaque nœud est externe car connecté proc./mem.

→ direct networks

(majorité interne)

Xbar : crossbar



in i out $k \Leftrightarrow$ routeur ligne i et colonne k
à passant

in $\neq$ et out $\neq \rightarrow$ pas de conflit

X bar ~~4096~~ $\times$ 4096

routeurs

16 M

liens

8 k bus

degré

2

(3 $\rightarrow$ 3)

long distance
ré-amplification

latence

2 bus $\times$ 4096

bisectional
bandwidth

4096

couche
métale

2

(simple à réaliser)

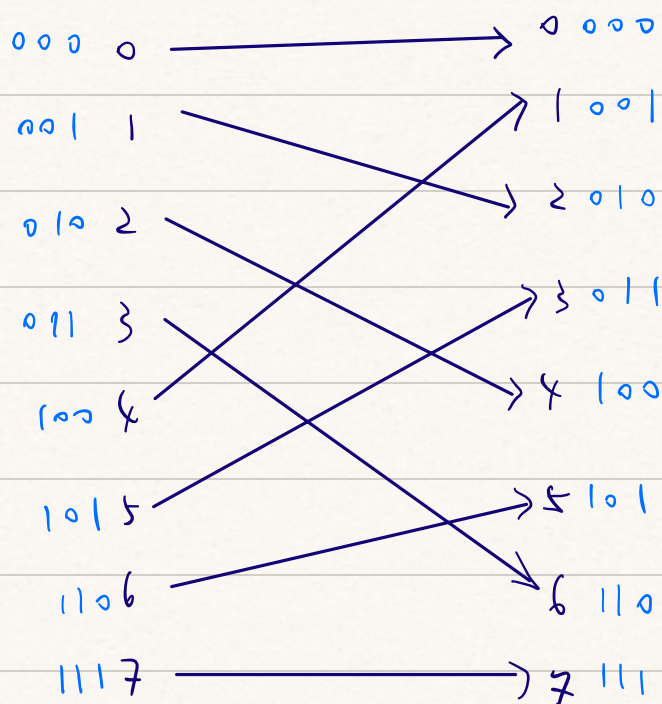
21/12/2023

Multi-stage

Interconnection

Network

perfect shuffle



$$i \in \llbracket 0, 3 \rrbracket \quad i \rightarrow 2i$$

$$i \in \llbracket 4, 7 \rrbracket \quad i \rightarrow 2(i-4) + 1$$

2^N

binairie

$$i \in \llbracket 0, 2^{N-1} - 1 \rrbracket$$

$$i \rightarrow 2i$$

$$i \in \llbracket 2^{N-1}, 2^N - 1 \rrbracket$$

$$i \rightarrow 2(i - 2^{N-1}) + 1$$

$$\sigma^N = \text{id}$$

$$i = (i'_{N-1}, \dots, i'_0)$$

$$i = (0, i'_{N-2}, \dots, i'_0)$$

$$2i = (i'_{N-2}, \dots, i'_0, 0)$$

$$i = (1, i'_{N-2}, \dots, i'_0)$$

$$\downarrow -2^{N-1} \quad (0, i'_{N-2}, \dots, i'_0)$$

$$\downarrow \times 2 \quad (i'_{N-2}, \dots, i'_0, 0)$$

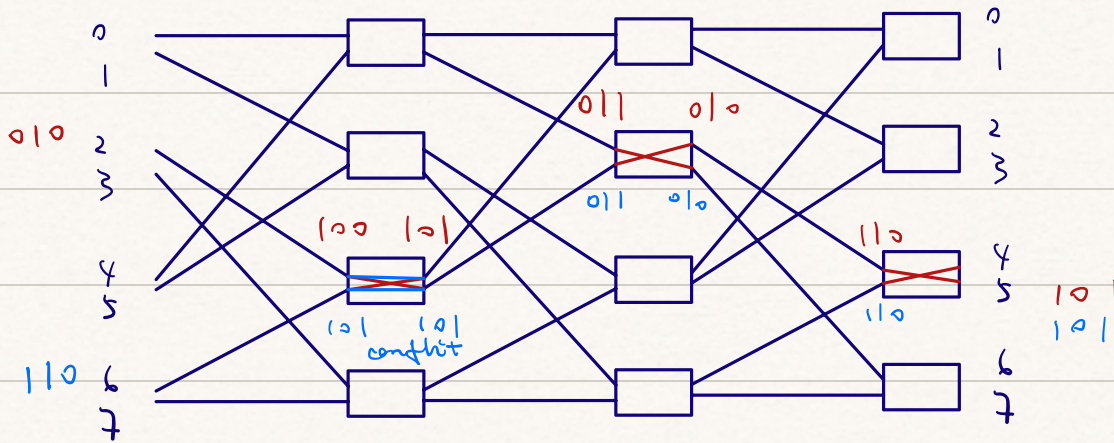
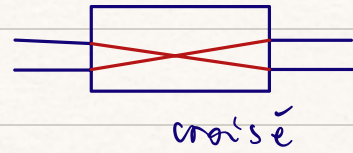
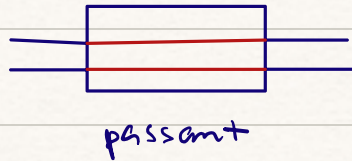
$$\downarrow +1 \quad (i'_{N-2}, \dots, i'_0, 1)$$

réseau Ω

8 in 8 out

3 étages

2 commutateurs



$s_2 s_1 s_0$

$\longrightarrow$

$d_2 d_1 d_0$

$\searrow s_1 s_0 s_2 = s_1 s_0 d_2 \longrightarrow s_0 d_2 s_1 = s_0 d_2 d_1 \longrightarrow d_2 d_1 s_0 = d_2 d_1 d_0$

$$\begin{aligned} & \overline{s_2 = d_2} \\ & \times \\ & s_2 \neq d_2 \end{aligned}$$

$$\begin{aligned} & \overline{s_1 = d_1} \\ & \times \\ & s_1 \neq d_1 \end{aligned}$$

$$\begin{aligned} & \overline{s_0 = d_0} \\ & \times \\ & s_0 \neq d_0 \end{aligned}$$

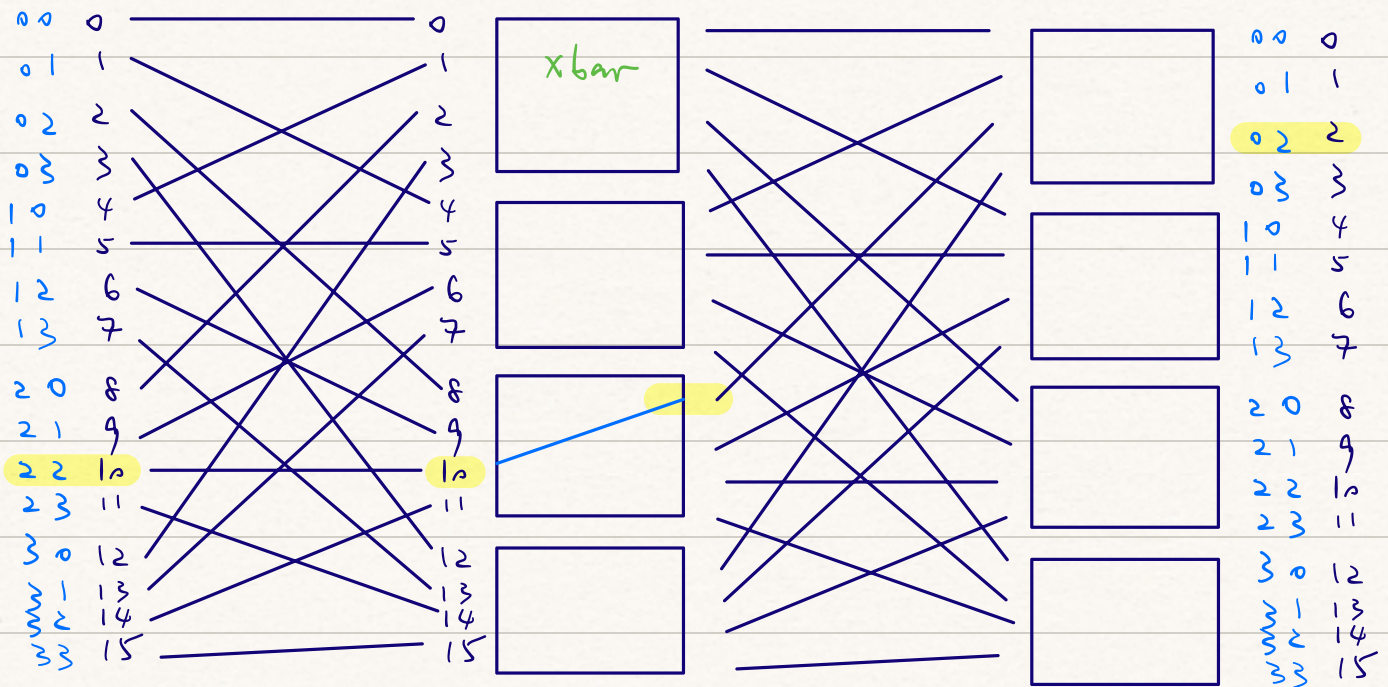
$\left. \begin{matrix} 2 \rightarrow 5 \\ 6 \rightarrow 4 \end{matrix} \right\}$ not simultaneous by

$\left. \begin{matrix} 0 \rightarrow 0 \\ 2 \rightarrow 5 \\ 7 \rightarrow 7 \end{matrix} \right\}$ simultaneously OK

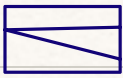
	hypercube 8	hypercube 2^k	Σ 8	$\Sigma 2^k$
routeurs	8	$2^k \times \frac{k}{2}$	12	$k 2^{k-1}$
liens	32	$k 2^{k-1} \times 2$	24	$k 2^k$
degré	3	k	4	$4 \times \frac{k}{4}$
latence	3	k	3	k
bisectional bandwidth	4	2^{k-1}	8	$2^k \times 2$
couche matérielle	? 2 1	? $k-1$ $k-3$	2	2

$$(s_1, s_0) \rightarrow (d_1, d_0) \quad s_i, d_i \in \llbracket 0, 3 \rrbracket$$

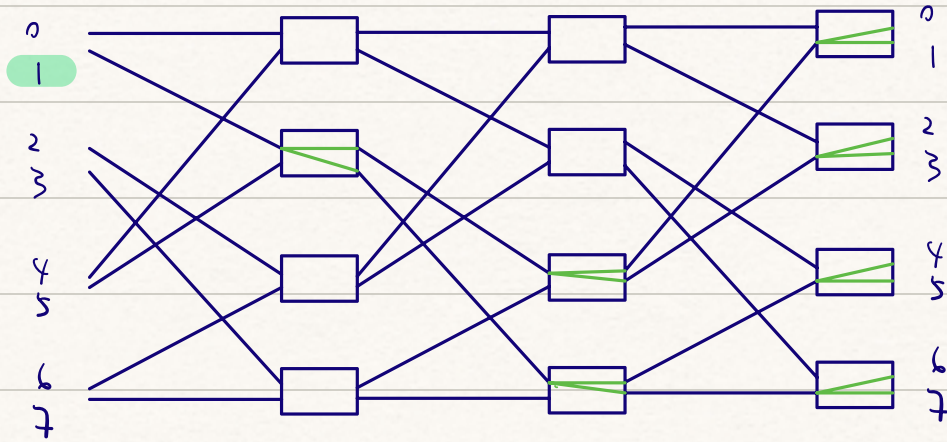
2 étages



Σ 16x16 4 étages

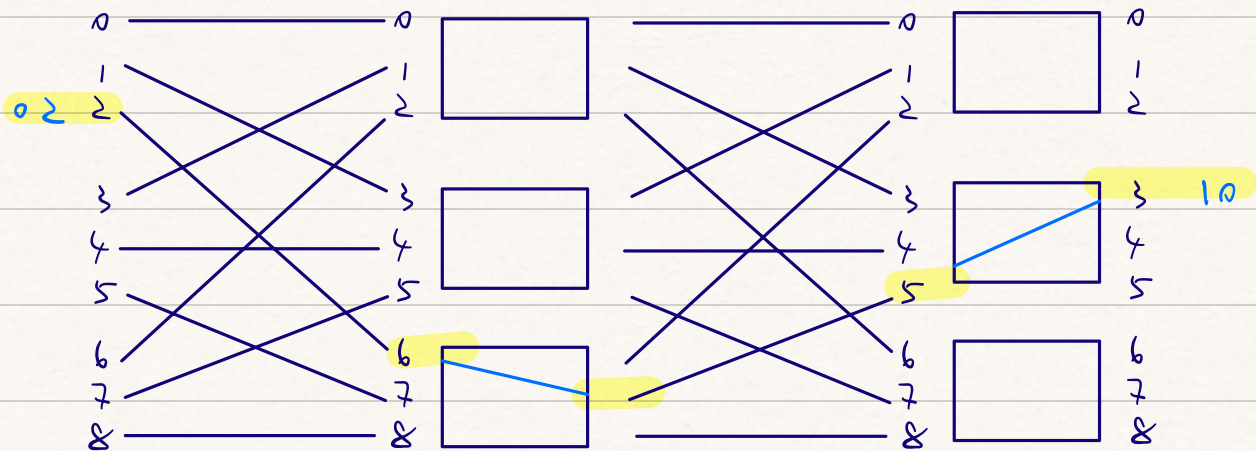


made diffusion



$$(s_1, s_2) \rightarrow (d_1, d_2)$$

$$s_i, d_i \in \llbracket 0, 2 \rrbracket$$



$$\text{benes } \Omega \rightarrow \Omega^R \text{ (renversé)}$$